Foundations in Data Engineering

Notes by Marco Galletti

Lecturer: Jana Giceva

Technical University of Munich
Winter Semester 2026/27

Contents

1 Basic building blocks: data formats and the command line **3**
- 1.1 Data exchange formats . . . 3
- 1.2 Streams and pipes . . . 4
- 1.3 Command-line tools as relational operators . . . 4

2 Performance and efficiency on a single machine **7**
- 2.1 Efficient baselines and hardware trends . . . 7
- 2.2 The memory hierarchy and locality . . . 9
- 2.3 Data layout in memory . . . 12
- 2.4 Virtual memory and address translation . . . 14

3 Anatomy of a simple query: language, compiler and I/O **16**
- 3.1 A toy query and its limits . . . 16
- 3.2 Interpreted and compiled code . . . 17
- 3.3 Reading a file: page cache, mmap and direct I/O . . . 18

4 Parallelism: SIMD and multithreading **20**
- 4.1 Why one thread is not enough . . . 20
- 4.2 Data-level parallelism: SIMD . . . 21
- 4.3 Task-level parallelism . . . 25
- 4.4 Measuring parallel performance . . . 26

5 The relational model, basic SQL and query execution **27**
- 5.1 Data models and data independence . . . 28
- 5.2 Relational operators and SQL semantics . . . 29
- 5.3 From SQL to physical execution . . . 30

6 Advanced SQL: aggregation and query composition **32**
- 6.1 Multiple grouping levels and statistical aggregates . . . 32
- 6.2 Nested queries, decorrelation and LATERAL . . . 34
- 6.3 Strings, conditional expressions and sampling . . . 36

7 Advanced SQL: common table expressions and window functions **36**
- 7.1 Common table expressions . . . 36
- 7.2 Recursive common table expressions . . . 37
- 7.3 Window functions . . . 39

A Command reference **43**

B Performance tools **44**

C SIMD and multithreading **45**

D SQL reference **46**

1 Basic building blocks: data formats and the command line

Given a large, unknown dataset, the first questions are what the data look like, which tool suits the analysis, and what hardware it requires. This section covers how datasets are encoded for exchange, and how the standard Unix command-line tools give a first impression of a dataset and answer simple queries on it without writing a program.

1.1 Data exchange formats

Datasets are exchanged as plain text (CSV and its variants), as text encoding semi-structured data (XML, JSON), in open binary formats such as **Parquet**, or in proprietary binary formats; the latter are poorly suited to exchange, since only the vendor's software reads them. Faced with an unknown file, the tool `file` guesses its format from its content. Below, the same two records (name, address, job) are written in each text format.

CSV. A **CSV** (comma separated values) file stores one record per line, its fields separated by a delimiter. The delimiter is configurable: with a tab one gets **TSV**. The format is simple, human readable and extremely widespread, but full of subtleties because it is weakly standardised: a field that contains the delimiter must be quoted, and a quote character inside a quoted field is doubled (the value `a"b` is written `"a""b"`). Records are flat: a row of fields, with no nesting.

```
Sherlock Holmes,221B Baker Street,Detective
James Moriarty,Reichenbach Falls,Villain
```

Written as "surname, first name", the first field would contain the delimiter and need quotes:

```
"Holmes, Sherlock",221B Baker Street,Detective
```

XML. **XML** encodes (semi-)structured data as a tree of nested elements delimited by opening and closing tags, possibly with attributes. In the example a root element contains one empty element `<person ... />` per record, whose fields are attributes:

```
<characters>
<person name="Sherlock Holmes" address="221B Baker Street" job="Detective"/>
<person name="James Moriarty" address="Reichenbach Falls" job="Villain"/>
</characters>
```

XML is better standardised than CSV and handles nested data, but it is very verbose (every value is wrapped in markup), human readable yet tedious to write, and its full specification is very complex. Advanced features such as **XML Schema** (a description of the allowed document structure) exist but are rarely used; its popularity is declining.

JSON. **JSON** (JavaScript Object Notation) is a much simpler text format. A JSON document is always an **array** `[v1, v2, ...]` or an **object** `{"key": value, ...}`: items are separated by commas, keys are strings in double quotes followed by a colon and a value, and values may again be arrays or objects, so data can be nested. There are only six data types: string, number, boolean, null, object and array. The two records form an array of two objects:

```
[{"name": "Sherlock Holmes", "address": "221B Baker Street", "job": "Detective"},
 {"name": "James Moriarty", "address": "Reichenbach Falls", "job": "Villain"}]
```

JSON is less verbose and easier to write than XML, and increasingly popular.

Remark. The most common JSON errors violate exactly these rules: unquoted keys, strings in single instead of double quotes, and a trailing comma after the last item of an array or object.

Format	Encoding	Structure	Verbosity	Standardisation and use
CSV/TSV	text	flat rows	low	weak, many subtleties; very popular
XML	text	nested	very high	well standardised, complex spec; declining
JSON	text	nested	moderate	simple spec; growing

Table 1: The text-based exchange formats compared.

1.2 Streams and pipes

Text files are popular because they are human readable and easy to process. The Unix command-line tools are available by default on every Unix/Linux system; each is simple, but they are surprisingly powerful in combination. They are ideal to get a first impression, to explore the data and to run simple analyses without code; for the real analysis one usually moves on to a more suitable tool.

Standard streams. Every process has three standard streams, identified by number: **stdin** (0) for input, **stdout** (1) for output and **stderr** (2) for error messages. A **pipe** feeds the stdout of one command into the stdin of the next, and **redirection** connects a stream to a file instead (fig. 1); the **null device** `/dev/null` discards what is written to it, so redirecting stderr there suppresses error messages. Since errors travel on their own stream, they do not mix with the data flowing through a pipeline. Most tools read the files given as arguments, or stdin if there are none.

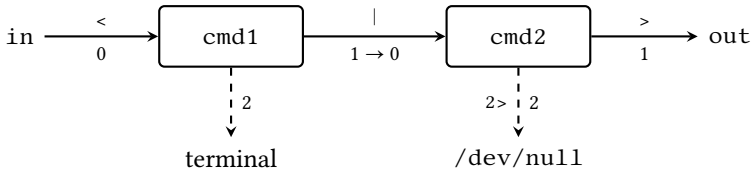


Figure 1: Streams of `cmd1 < in | cmd2 2> /dev/null > out`; stderr of `cmd1` still goes to the terminal.

Streaming large and compressed files. A tool in a pipeline sees its input as a stream of lines, so files are processed without being loaded or copied. In particular, the `*cat` variants of the compressors (gzip: Lempel–Ziv; bzip2: Burrows–Wheeler transform; xz: LZMA2) decompress to stdout, and a compressed dataset can be analysed without ever writing the decompressed data to disk. Pagers and prefix/suffix tools (`less`, `head`, `tail`) and counts (`wc`) give the first impression of a file: what a record looks like, how many there are.

1.3 Command-line tools as relational operators

A line-based text file can be read as a relation: each line is a tuple, and the fields separated by a delimiter (whitespace, a comma, a tab) are its attributes. The tools then act as relational operators on a stream of tuples, and a pipeline composes them into a query (table 2 and fig. 2).

Operator	Tool
selection (keep the lines satisfying a predicate)	`grep` (regular expression), `grep -v` (negated)
projection (keep some attributes)	`cut` (fields), `grep -o` (matched part)
sorting, duplicate elimination	`sort`, `uniq`
grouping with count	`sort`, then `uniq -c`
top-k, minimum, maximum	`sort`, then `head` or `tail`
join (inner, outer, anti)	`join` on sorted inputs
map (rewrite each line)	`sed`
aggregation (count, sum, average)	`wc -l`, `awk`
random sample	`shuf`, then `head`

Table 2: Command-line tools seen as relational operators; their flags are collected in section A.

Selection with regular expressions. A **regular expression** (regex) describes a set of strings. `grep` keeps the lines that contain at least one substring matched by the regex (selection), `grep -v` the lines that contain none (negated selection), and `grep -o` prints only the matched substrings, one per line (projection onto a pattern-defined part of the line); `sed` uses the same syntax for substitutions. A regex is built from the following pieces (table 3): ordinary characters match themselves; metacharacters have a meaning and match literally only when escaped with a backslash; a **quantifier** applies to the single item just before it (a character, a class or a group).

	Regex	Matches
characters	`a`, `.`, `\.`	the letter `a`; any single character; a literal dot
classes	`[abc]`, `[a-z0-9_]`	one character from the set (ranges allowed)
	`[^abc]`	one character *not* in the set
	`[[:space:]]`, `[[:digit:]]`	one whitespace character; one digit
quantifiers	`r*`, `r+`, `r?`	≥ 0, ≥ 1, 0 or 1 repetitions of `r`
	`r{n}`, `r{n,}`, `r{n,m}`	exactly n, at least n, between n and m
anchors	`^`, `$`	start, end of the line (match no character)
groups	`(r)`, `r|s`, `\1`	group; `r` or `s`; the text matched by group 1

Table 3: Regular expression syntax (extended form).

Basic and extended syntax, greediness. `grep` and `sed` use the **basic** syntax by default, in which `+ ? { } ( ) |` are ordinary characters and the operators must be escaped (`\{n,\}`, `\(r\)`; GNU also accepts `\+ \? \|`); `grep -E` (and `sed -E`) switch to the **extended** syntax of table 3. A regex is not anchored: `\.jpg` matches any line containing `.jpg`, and `^` or `$` are needed to pin it to the start or end. Quantifiers are **greedy**, i.e. they take the longest possible match, so `user=.*$` extracts everything from `user=` to the end of the line.

Remark. On the command line the regex is put in single quotes, so that the shell does not interpret `*`, `$` or `|` itself. The shell has its own patterns (**globs**) for file names, with a different meaning of `*`: the glob `*.html` ("any string, then `.html`") corresponds to the regex `^.*\.html$`, whereas in a regex `*` repeats the item before it.

Example 1.1 (Reading regexes)**.** `img[0-9]*\.jpg` matches `img` followed by zero or more digits and `.jpg`: it matches the lines `img.jpg`, `img7.jpg`, `img123.jpg`, and also `myimg12.jpg.bak` (the regex is not anchored), but not `imgA.jpg` (a letter is not a digit) nor `img5xjpg` (`\.` requires a dot). The regex `^warning` with `grep -v` keeps the lines that do not start with `warning`.

Example 1.2 (Extracting e-mail addresses)**.** With `grep -o -i -E`, the regex

$$\underbrace{\texttt{[_.a-z0-9]+}}_{\text{local part}}\ \texttt{@}\ \underbrace{\texttt{[a-z0-9]+}}_{\text{domain}}\ \underbrace{\texttt{\textbackslash.}}_{\text{dot}}\ \underbrace{\texttt{[a-z]\{2,\}}}_{\text{top-level domain}}$$

prints each e-mail address of the input on its own line: one or more letters, digits, underscores or dots, then `@`, one or more letters or digits, a literal dot and at least two letters (`-E` because of `+` and `{2,}`, `-i` so that the classes also cover capital letters). On `Ada.Lovelace@Example.ORG` it returns the whole address. It is a simplification of the full standard: the domain class contains neither dots nor hyphens, so from `x@in.tum.de` it extracts only `x@in.tum`, and `foo@mail-server.com` is not found at all.

Order matters. `sort` compares lexicographically unless told to compare numerically (so `10` comes before `9`), and it works on inputs larger than main memory. Its output is the required input of `uniq` and `join`: `uniq` only merges *adjacent* equal lines, and `join` merges two files by scanning both in order. Grouping is therefore sort-based: sorting brings equal values together, then `uniq` counts each run. The lines "occurring exactly once" (`uniq -u`) are not the distinct lines (plain `uniq`).

Top-k and frequencies. A **top-k query** is a sort followed by taking a prefix or a suffix; $k = 1$ gives the minimum or the maximum. Composed with grouping it yields the k most frequent values of an attribute (fig. 2).

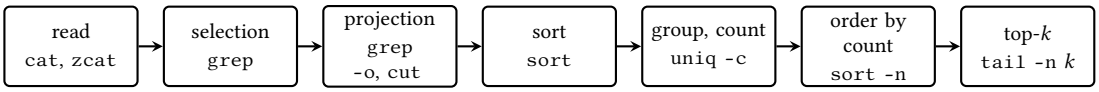


Figure 2: A pipeline computing the k most frequent values of an attribute, stage by stage.

Join. `join` pairs the lines of two sorted files with equal join field (an inner join). Keeping in addition the unmatched lines of the first, the second or both files gives the left, right and full **outer join**; keeping *only* the unmatched lines of one file gives the **anti-join**, which answers "which values of one list never occur in another". For heavier use of joins a more powerful tool is the better choice.

Map and aggregation. `sed` rewrites each line by substituting the matches of a regex; the replacement may refer to the whole match (`&`) or to the groups of the regex (`\1`, `\2`, …). For instance, the substitution (basic syntax)

```
s/IMG_\([0-9]*\)\.JPG/image\1.jpg/
```

renames `IMG_0042.JPG` to `image0042.jpg`. Replacing whitespace by newlines puts one word per line, which reduces word-level questions (e.g. word frequencies) to the line-level operators above. `awk` runs a small program in one pass: an initialisation, a block executed on every line (with access to its fields), and a final block. Writing a_i, b_i for the first two fields of line $i = 1, \dots, n$, this computes aggregates such as

$$n, \qquad \sum_{i=1}^{n} b_i, \qquad \frac{1}{n}\sum_{i=1}^{n} b_i, \qquad \sum_{i\colon a_i > 10} b_i,$$

by accumulating a sum and a count in variables and printing them (or their ratio) at the end.

Sampling. `shuf` outputs a random permutation of its input, so a prefix of it is a uniform random sample without replacement. A plain prefix of the file is not: files are often ordered (by time, by key), so their first lines are not representative. Shuffled inputs also serve for performance tests.

Limits. These tools stream their input and `sort` works beyond main memory, so they scale to large files and are ideal for a first look and simple queries. They operate on lines of text, however: they know nothing about quoting (`cut` splits a quoted CSV field containing the delimiter) or nesting, and anything beyond simple joins and aggregates calls for more powerful tools.

2 Performance and efficiency on a single machine

The tools of section 1 reveal the structure of raw data through small composable transformations. The next question is when a more powerful tool, or more hardware, is needed: what performance a single machine delivers, how far high-level tools are from hand-written code, what the hardware limits are, and how large a cluster would have to be to do better. This section reviews the hardware trends and the memory hierarchy, and shows how the order of the accesses and the layout of the data in memory change the running time without changing the algorithm.

2.1 Efficient baselines and hardware trends

Efficient baselines. Many big-data systems are evaluated only by their **scalability**, the speed-up obtained by adding cores or machines, measured against the same system on one core instead of a good single-threaded implementation. This can mislead: a system A may reach a speed-up of about 50 on 300 cores while a system B levels off around 5, and still B is faster than A at every core count, because A is much slower to begin with. Scaling out is the right choice when it is needed, but only after a careful implementation on one machine ("you can have a second computer once you've shown you know how to use the first one", P. Barham).

Remark. The paper *Scalability! But at what COST?* (McSherry, Isard, Murray, 2015) calls COST of a system the configuration (number of cores) it needs to outperform a competent single-threaded implementation.

Example 2.1 (PageRank on one thread)**.** Table 4 reports the time of twenty iterations of PageRank on two graphs (the Twitter follower graph and the web graph uk-2007-05) for published graph-processing systems running on 2 to 128 cores, and for a careful single-threaded implementation. On Twitter only GraphLab (128 cores) beats the single thread; on uk-2007-05 only GraphX (128 cores) does. If the data fit on the SSD or in the DRAM of one machine, scaling beyond it may not pay off.

System	Cores	Twitter	uk-2007-05
GraphChi	2	3160 s	6972 s
Stratosphere	16	2250 s	–
X-Stream	16	1488 s	–
Spark	128	857 s	1759 s
Giraph	128	596 s	1235 s
GraphLab	128	249 s	833 s
GraphX	128	419 s	462 s
single thread (SSD)	1	300 s	651 s
single thread (RAM)	1	275 s	–

Table 4: Twenty iterations of PageRank: scalable systems against one thread.

End of Dennard scaling. For decades the number of transistors per chip grew exponentially (**Moore's law**) and single-threaded performance doubled every 18 to 22 months. **Dennard scaling** states that as transistors get smaller their power density stays constant, so that power consumption stays proportional to chip area. As long as it held, every generation packed more transistors and ran them at a higher clock frequency for about the same power. It broke down around 2004, when voltage could no longer be scaled down together with transistor size: from then on a higher frequency means more power, hence more heat, than a chip can dissipate (the **power wall**). Since then clock frequency and typical power have plateaued, Moore's law has slowed down, single-threaded performance stagnates, and the extra transistors go into more parallelism instead, especially on servers. Three effects follow.

Effect 1: parallelism. The industry switched from making a single core faster to adding cores. In AMD server processors the maximum number of physical cores went from 1 in 2003 to 32 with the first EPYC (2017) and 256 in 2026. Software used to rely on **instruction-level parallelism** (ILP), which hardware and compiler exploit implicitly, without the programmer's help. The parallelism offered now is explicit: **thread-level parallelism** (TLP, many cores) and **data-level parallelism** (DLP). Exploiting TLP and DLP requires restructuring the application; this is the topic of the next lecture.

SIMD. Within general-purpose processors DLP takes the form of **SIMD** instructions (single instruction, multiple data): one instruction applies the same operation to all the values packed in a vector register. The width of these registers has grown steadily, as Intel's extensions show (fig. 3): MMX, 64 bits (1997); SSE, 128 bits (1999); AVX and AVX2, 256 bits (2011 and 2013, for floating point and integers); AVX-512, 512 bits (2017). Vector processors and GPUs push the same idea further and are increasingly popular.

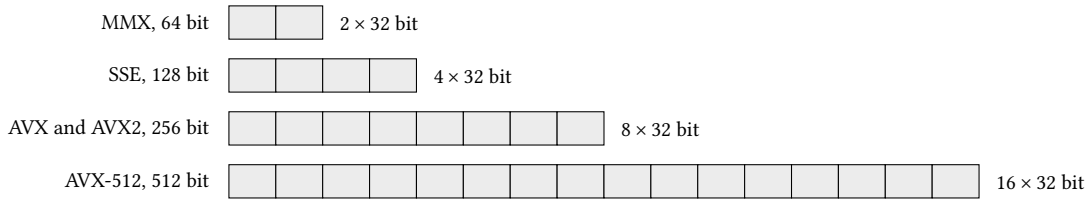


Figure 3: Register width of Intel's SIMD extensions, as the number of 32-bit values one instruction processes.

Effect 2: the memory wall. Memory bandwidth grows more slowly than the core count, so the DRAM bandwidth available per core has decreased: on Intel server processors it fell from about 12 GB/s in the early 2000s to 3–4 GB/s around 2016, and recent AMD EPYC generations offer a theoretical maximum of only 3–5 GB/s per core. Meanwhile the cost of computation (per GFLOP/s) has fallen much faster than the cost of DRAM and disk capacity (per TB). Memory has become "the new disk": the gap between processor and DRAM now plays the role that the gap between DRAM and disk used to play, so processors need a multi-level cache hierarchy, and programs must use registers and caches carefully (section 2.2). Caches bring their own pitfalls: cache pollution, the coherence protocol between the caches of different cores (MESI), timing attacks.

Effect 3: hardware specialisation. Further gains come from **domain-specific architectures** (DSAs), hardware specialised for one class of tasks: GPUs, TPUs, NPUs, FPGAs, ASICs. Specialised hardware was the norm before general-purpose processors took over (database machines, Lisp machines) and is spreading again: special instructions (AVX-512, AMX for matrices), AI accelerators and NPUs, SmartNICs and DPUs (network cards that run code), computational storage (smart SSDs), FPGAs, and on-chip accelerators that take memory copies, (de)compression and scan/filter off the cores. The trend towards **active hardware** processes data where it resides or while it moves: smart storage and NICs, programmable switches, processing in or near memory, smart DMA engines, perhaps programmable memory controllers.

Interconnects. **CXL** (Compute Express Link) is an open cache-coherent interconnect between CPUs, accelerators and memory devices. It distinguishes accelerators without local memory (type 1), accelerators with local memory such as GPUs, ASICs and FPGAs (type 2), and memory expansion devices (type 3); CXL 3.x moves towards fabric-attached memory with memory pooling and sharing. Networks have low latency and high bandwidth: InfiniBand in high-performance computing, 400–800 Gbit/s Ethernet already deployed in data centres, 1.8 Tbit/s emerging.

A single machine today. A server CPU of 2026 (AMD EPYC, 6th generation) has up to 256 cores (512 threads) and a 1 GB L3 cache; a two-socket server holds up to 4 TB of DRAM, with 1–1.6 TB/s of memory bandwidth per socket.

2.2 The memory hierarchy and locality

Memory pyramid. Storage technologies trade capacity for speed: the closer to the CPU, the smaller, faster and more expensive per byte (fig. 4). Registers hold 8-byte words; three levels of on-chip SRAM cache follow (L2 used to be off-chip), the third being the **last-level cache** (LLC); then main memory (DRAM), SSDs and hard disks. Other memory technologies offer more bandwidth than DRAM: up to 1 TB/s for GDDR and 3–5 TB/s for HBM.

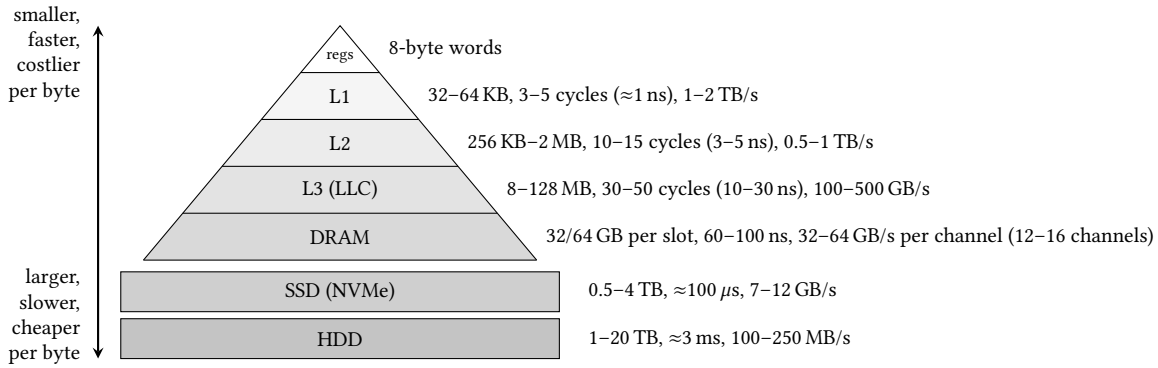


Figure 4: The memory hierarchy with typical capacity, latency and bandwidth of each level.

Latency numbers. Table 5 extends the hierarchy to storage, network and other frequent operations (a list originally compiled by Jeff Dean). Between 2012 and 2026 bandwidth improved dramatically (sequential SSD bandwidth from about 1 to 14 GB/s, DRAM bandwidth by up to a factor of 10, depending on the platform), whereas latency often did not. These numbers allow back-of-the-envelope estimates of the running time to expect, and show what to keep off the critical path (the hot loop).

Operation	2012	2026
CPU cycle	0.3 ns	0.2 ns
L1 cache reference	0.5 ns	1 ns
branch mispredict	5 ns	3–5 ns
L2 cache reference	7 ns	3 ns
L3 cache reference	–	10–20 ns
main memory (DRAM) access	100 ns	80–120 ns
read 1 MB sequentially from memory	250 µs	10–30 µs
SSD random read (4 KB)	150 µs	50–100 µs
read 1 MB sequentially from SSD	1 ms	35–100 µs
disk seek (random access)	10 ms	5–10 ms
read 1 MB sequentially from disk	20 ms	4–10 ms
round trip within a data centre	500 µs	< 1 ms
intercontinental round trip	150 ms	140–160 ms

Table 5: Latency numbers, 2012 and representative 2026 values.

Example 2.2 (Back-of-the-envelope estimates)**.** Scaling the 1 MB rows of table 5 by 1000, reading 1 GB sequentially takes about 10–30 ms from DRAM, 35–100 ms from an SSD and 4–10 s from a hard disk. Reading the same 1 GB as random 4 KB blocks from the SSD takes $2^{30}/2^{12} = 2^{18} \approx 2.6 \cdot 10^5$ random reads, i.e. $2.6 \cdot 10^5 \times (50\text{–}100\ \mu s) \approx 13\text{–}26$ s if they are issued one after the other: hundreds of times slower than the sequential read.

Cache lines. Caches move data in fixed-size units, the **cache lines** (typically 64 bytes): a miss loads a full line and an eviction removes a full line. A program should therefore use everything it brings into a line (fig. 5). A hardware **prefetcher** monitors the access pattern and, when it recognises a sequential or strided pattern, speculatively requests the next lines before the program loads from them.

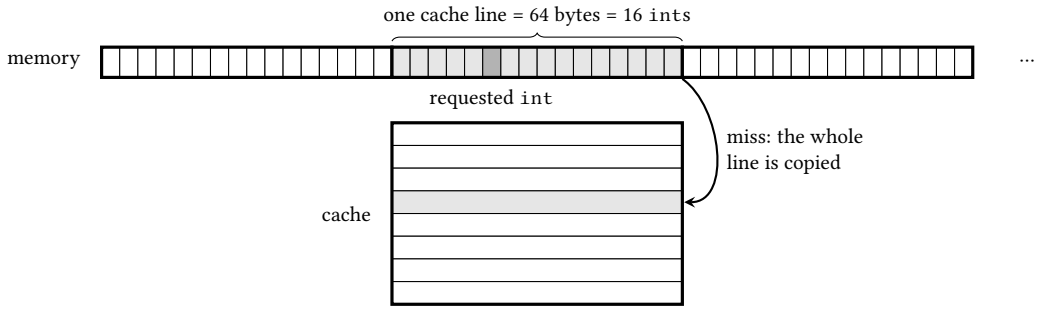


Figure 5: A cache of 8 lines: a miss on one `int` loads the whole 64-byte line that contains it.

Locality. Caches pay off thanks to the **principle of locality**. **Spatial locality**: data close to recently accessed data are likely to be accessed soon (arrays traversed with stride 1, instructions executed in sequence). **Temporal locality**: recently accessed data and code are likely to be accessed again (a function called repeatedly, a hot set of data that fits in the cache). A cache exploits both: it keeps recently used lines (temporal locality) and loads whole lines, plus the next ones through the prefetcher (spatial locality). The **stride** of an access pattern is the distance, in elements, between consecutive accesses.

Example 2.3 (Locality of a loop)**.** In the loop

```
sum = 0; for (i = 0; i < n; i++) sum += a[i]; return sum;
```

the variable `sum` is referenced in every iteration and the loop body is executed repeatedly (temporal locality of data and of instructions); the array `a` is read with stride 1 and the instructions are fetched in sequence (spatial locality of data and of instructions).

Traversal order. A two-dimensional C array `int a[M][N]` is stored in **row-major** order, row after row, so that $a[i][j]$ lies at address base + $4(iN + j)$ (an `int` has 4 bytes). The two functions below compute the same sum but visit the elements in different orders.

```
/* sum_array_rows */
for (i = 0; i < M; i++)
  for (j = 0; j < N; j++)
    sum += a[i][j];
```

```
/* sum_array_cols */
for (j = 0; j < N; j++)
  for (i = 0; i < M; i++)
    sum += a[i][j];
```

The row-wise loop visits consecutive addresses (stride 1); the column-wise loop jumps by a whole row at every step (stride N, i.e. $4N$ bytes), see fig. 6. With 64-byte lines the row-wise loop uses all 16 `int`s of each line it loads, so it misses at most once every 16 accesses, and the prefetcher recognises the pattern. The column-wise loop uses only one `int` of each line it loads; the other 15 belong to the next columns, but before the loop comes back to the same line it touches one line in every row, M lines in total. When they do not fit in the cache (for $M = 20\,000$ they occupy $20\,000 \times 64$ bytes ≈ 1.3 MB, more than L1 and L2) each line is evicted before it is reused, and almost every access misses in these caches.

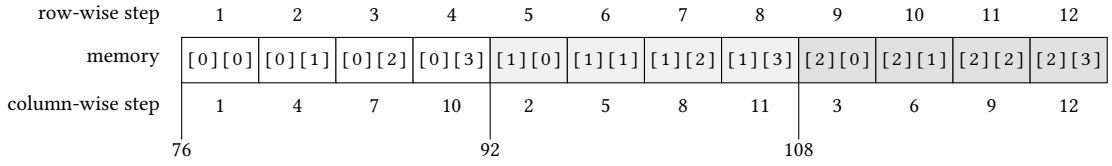


Figure 6: `int a[3][4]` in memory (addresses from 76) and the step at which each loop reads each element.

For a $20\,000 \times 20\,000$ matrix the column-wise function takes 16.98 s and the row-wise one 1.71 s, an order of magnitude; the profiler `perf` attributes about 76% of the CPU cycles and 87% of the cache misses of the program to the column-wise function. In a second experiment the number of elements is fixed at 10^9 and the shape varies from one row to one column: the row-wise loop takes about 2 s for every shape, while the column-wise loop is up to 50 times slower. Its working set is the set of lines it keeps reusing, one per row. With few rows these lines stay in the cache and the next columns find them there; as the number of rows grows the working set overflows L1, L2 and L3 in turn and the time rises. With fewer than 16 columns a line holds several consecutive rows, so even the column-wise loop reuses each loaded line more and more, and the gap disappears for a single column.

The memory mountain. The read throughput of a loop, measured as a function of the working-set size and of the stride, forms a surface with two kinds of features. Along the size axis throughput drops in steps, the **ridges of temporal locality**: one plateau for each level (L1, L2, L3, main memory) that holds the working set. Along the stride axis throughput decreases as the stride grows, the **slopes of spatial locality**, since fewer bytes of each loaded line are used. At stride 1 aggressive prefetching keeps throughput high even for large working sets (fig. 7). On a desktop processor with 32 KiB L1, 256 KiB L2 and 8 MiB L3 the peak (small working set, stride 1) is around 14 GB/s, more than ten times the throughput of large strides on working sets that only fit in main memory.

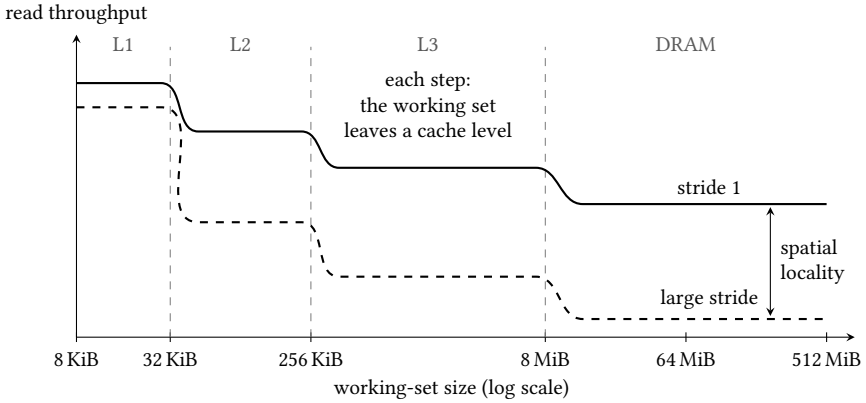


Figure 7: A cut of the memory mountain for two strides (schematic, illustrative values).

Cache-friendly code. The programmer controls how the data are organised (layout and alignment, section 2.3) and how they are accessed (the nesting of the loops, and **blocking**, i.e. processing the data in blocks that fit in the cache). Most of the benefit comes from generic rules: keep the working set small (temporal locality), use small strides (spatial locality), and focus on the innermost loop. The same considerations apply beyond caches:

- access pattern: sequential access is always better than random access, for DRAM as for hard disks;
- layout: the arrangement of the data in memory or storage should suit the workload, e.g. in databases, row stores for transactional workloads (OLTP: many short transactions that read or update whole records) and column stores for analytical ones (OLAP: queries that scan a few attributes of many records), see section 2.3;
- format: human-readable formats may not deliver the required performance; converting the data into an efficient format is one reason for **ETL** pipelines (extract, transform, load), more recently **ELT**, both treated later in the course.

Tools differ in how much control they give over these aspects.

2.3 Data layout in memory

Structures. A C `struct` is stored as one contiguous block of memory large enough for all its fields, which are placed in declaration order even when another order would be more compact. The compiler determines the size and the offset of every field; the machine code sees only addresses and offsets, not the structure. For instance `struct rec { int a[4]; long i; struct rec *next; }` occupies 32 bytes, with `a` at offset 0, `i` at 16 and `next` at 24.

Alignment. A primitive object of K bytes is **aligned** if its address is a multiple of K; for $K = 2^m$ this means that the lowest m bits of the address are zero (table 6). Memory is accessed in word-sized chunks, so loading or storing a value that spans a word boundary, and even more a cache-line boundary, is inefficient: an 8-byte `double` at address 60 crosses the line boundary at 64 and needs two lines (fig. 8); aligned data are therefore recommended for performance, although x86-64 hardware also works correctly on unaligned data.

K	Type	Address
1	`char`	no restriction
2	`short`	lowest bit zero
4	`int`, `float`	lowest 2 bits zero
8	`long`, `double`, pointers	lowest 3 bits zero
16	`long double`	lowest 4 bits zero

Table 6: Alignment of the primitive types on x86-64.

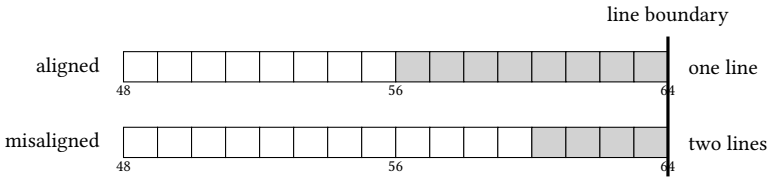


Figure 8: An 8-byte `double` at an aligned address (56) and at a misaligned one (60).

Padding. To respect alignment the compiler

- keeps the declared order of the fields;
- aligns each field within the struct, inserting padding before it if needed (`offsetof` gives the actual offset of a field);
- aligns the struct as a whole according to its largest field;
- makes the total size a multiple of that alignment, inserting padding at the end (`sizeof` gives the true size).

Padding wastes space (**internal fragmentation**), but the padded struct is faster to access than the packed one. Since the order is fixed, the programmer can often save space by declaring the large fields first.

Example 2.4 (Padding and reordering)**.** `struct S1 { char c; int i[2]; double v; }` packed would put `i` at offset 1 and `v` at 9, both misaligned (17 bytes). The compiler puts `i` at 4 (a multiple of 4) and `v` at 16 (a multiple of 8), after 3 and 4 bytes of padding: 24 bytes, a multiple of the struct's alignment 8. `struct S4 { char c; int i; char d; }` gets offsets 0, 4, 8, and its size is rounded up from 9 to 12, a multiple of 4. Declaring the `int` first, `struct S5 { int i; char c; char d; }` gets offsets 0, 4, 5 and size 8 (6 rounded up to a multiple of 4). See fig. 9.

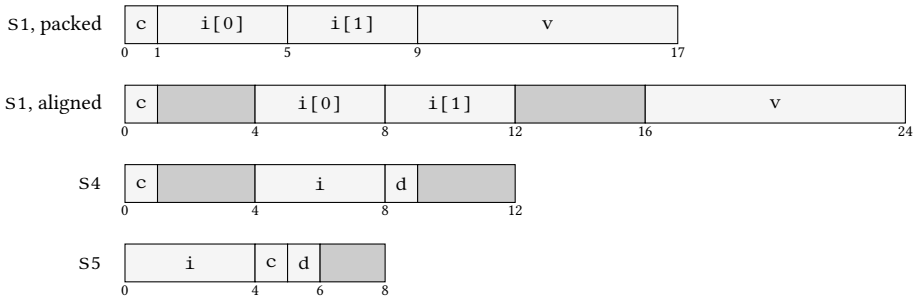


Figure 9: Byte layouts of `S1`, `S4` and `S5`; grey cells are padding.

The last two rules exist because of arrays: element k of an array of structs starts at k times the size of the struct, so every element, and with it every field, stays aligned only if the size is a multiple of the alignment. If `S4` had size 9, the second element would start at 9 and its `int` at 13 (fig. 10).

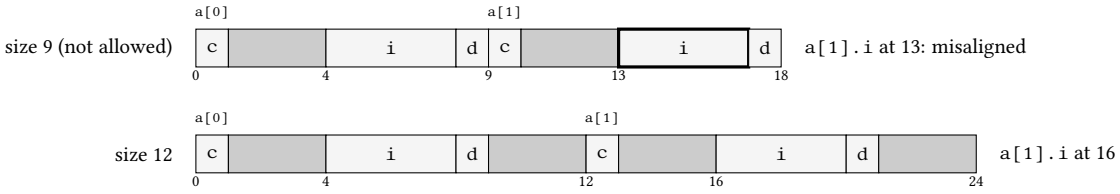


Figure 10: An array of two `S4`: without trailing padding the second `int` would be misaligned.

Variable-length data. Strings and other variable-length fields are split into a fixed-size header, which holds the length and a pointer to the data, and a variable-size tail placed at the end of the record and treated as having alignment 1, so that the fixed-size fields keep fixed, aligned offsets (fig. 11).

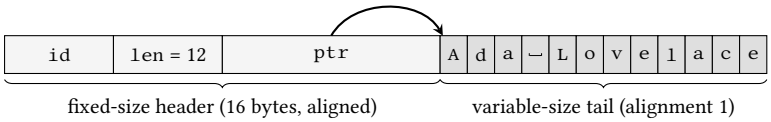


Figure 11: A record `(id, name)`: the header holds the length and a pointer to the string stored at the end.

Effect of alignment. Inserting tuples

```
struct { int primary_key; long timestamp; char color[2]; int zipcode; }
```

into an array (one socket, 4 hardware threads) reaches a throughput of 0.523 MB/s without alignment, 11.7 MB/s with padding, and 814.8 MB/s with reordered fields plus padding.

Array of structures and structure of arrays. A collection of records can be stored as an **array of structures** (AoS), each record contiguous in memory, or as a **structure of arrays** (SoA), one array per field (fig. 12); this is the distinction between a **row store** and a **column store**. An update that changes several fields of one record touches one place in AoS but several arrays in SoA; a scan of one field reads only that field's array in SoA, whereas in AoS every loaded cache line also carries the other fields.

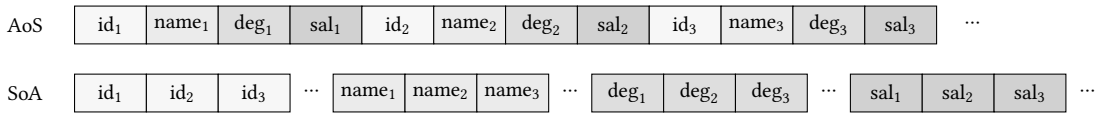


Figure 12: Three records `(id, name, degree, salary)` as an array of structures and as a structure of arrays.

Example 2.5 (AoS versus SoA)**.** On 100 million records `(id, name, degree, salary)`, changing the degree and the corresponding salary of 10 million records takes 126.58 ms in AoS and 195.66 ms in SoA, while summing all salaries takes 137.22 ms in AoS and 45.67 ms in SoA. AoS suits updates of whole records, as in OLTP (about 1.5 times faster here); SoA suits reads of few attributes, as in OLAP (3 times faster here).

2.4 Virtual memory and address translation

Each user-space process has its own **virtual memory**: the illusion of running alone on the machine with access to all of its memory. The addresses a program uses are **virtual addresses** (VA); the hardware translates them into **physical addresses** (PA) of the DRAM.

Pages. Virtual and physical memory are divided into blocks of fixed size, the **pages** (typically 4 KB), and the translation is defined page by page: each virtual page of a process is stored in some physical page of the DRAM, or is currently not in DRAM (kept on disk), or is unused. Consecutive virtual pages may sit anywhere in physical memory, and two processes may map one of their pages each to the same physical page (fig. 13). This provides

- isolation and protection: each process runs in its own virtual address space, which other processes can neither read nor overwrite;
- simpler memory management: the process sees a contiguous address space, while its physical pages may be scattered;
- efficient use of DRAM: the operating system can page data in and out of physical memory, keeping on disk the pages not in use and loading them back when needed;
- sharing: processes can safely share memory by mapping the same physical pages into their virtual address spaces (`mmap`, used in the next lecture).

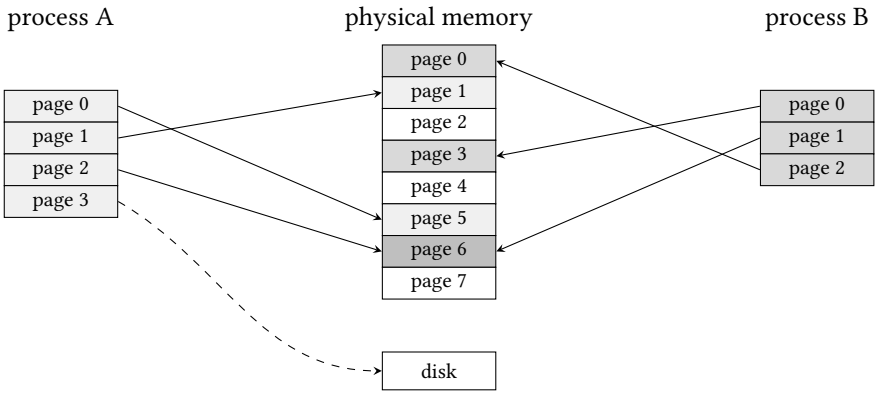


Figure 13: Virtual pages of two processes mapped to physical pages (illustrative mapping); the darkest physical page is shared by both, and one page of A is on disk.

Address translation. Since translation works page by page, an address is split into a page number and an **offset** within the page: with pages of 4 KB = 2^{12} bytes the lowest 12 bits are the offset and the remaining ones form the **virtual page number** (VPN). The **page table** (PT) of the process, kept in memory, has one **page table entry** (PTE) per virtual page, which contains the number of the physical page that holds it (PPN). The physical address is the PPN followed by the unchanged offset (fig. 14).

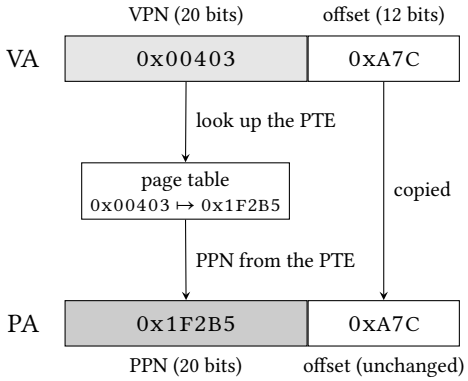


Figure 14: Translating a virtual address with 4 KB pages (32-bit addresses, illustrative values).

The translation is done by the **memory management unit** (MMU) on the CPU chip, in five steps:

1. the CPU sends the VA to the MMU;
2. the MMU computes the address of the PTE (PTEA: start of the page table plus VPN times the size of an entry) and requests it from cache or memory;
3. cache or memory returns the PTE;
4. the MMU forms the PA and sends it to cache or memory;
5. cache or memory returns the data to the CPU.

Every memory access thus requires a second access, to the page table, before the data can be read.

TLB. To avoid this extra access the MMU keeps a small cache of recent translations, the **translation lookaside buffer** (TLB), which maps VPNs to PTEs (fig. 15). On a **TLB hit** steps 2 and 3 are replaced by a lookup in the TLB, so the page table is not accessed at all. On a **TLB miss** the MMU fetches the PTE from cache or memory as above, stores it in the TLB, and then sends the PA. Hits are the common case thanks to locality: a 4 KB page holds 1024 `int`s, so a program that works on a few pages at a time needs only a few translations, and these stay in the TLB.

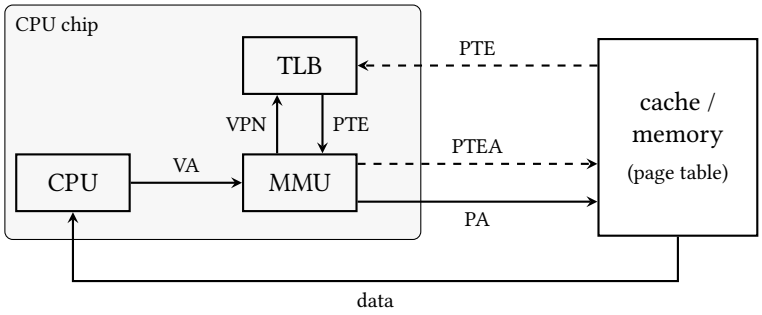


Figure 15: Address translation: solid arrows on a TLB hit; the dashed ones are added on a TLB miss.

Example 2.6 (TLB and traversal order)**.** Scanning an `int` array sequentially needs one new translation every 1024 elements, so at most one TLB miss per 4 KB page. In the column-wise traversal of a $20\,000 \times 20\,000$ `int` matrix consecutive accesses are one row, i.e. 80 000 bytes, apart, so each of them falls in a different page and needs its own translation, and one column touches 20 000 pages, far more than a TLB holds.

3 Anatomy of a simple query: language, compiler and I/O

One simple query, timed in several implementations, shows where the time of a data-processing program goes: into interpreting the code, into the work of the CPU, and into reading the file. This section follows that query from a first estimate of the hardware limits down to the way the operating system delivers file data.

3.1 A toy query and its limits

The query. The **TPC-H** benchmark generates a synthetic database whose size is set by a **scale factor** SF. Its relation `lineitem` has about $6 \cdot 10^6 \times \mathrm{SF}$ rows and takes 725 MB × SF as text (the whole database about 1 GB × SF); each row is a line with fields separated by `|`:

```
1|155190|7706|1|17|21168.23|0.04|0.02|N|0|1996-03-13|...
1|67310|7311|2|36|45983.16|0.09|0.06|N|0|1996-04-12|...
```

The query sums the fifth field (quantity) over all rows, at SF = 1. It is much simpler than the real benchmark queries, so it is easy to analyse, and yet surprisingly expensive.

Example 3.1 (Performance limits)**.** Before measuring anything, a rough estimate tells how fast the query can possibly be: no program can be faster than the time needed to deliver the 725 MB to the CPU, i.e. the size divided by the bandwidth of the source (with 1 MB = 10^6 bytes and 1 Gbit/s = 125 MB/s):

Source	Bandwidth	Minimal time
1 Gbit/s Ethernet	125 MB/s	5.8 s
HDD	200 MB/s	3.6 s
SATA SSD	500 MB/s	1.45 s
NVMe SSD	2 GB/s	0.36 s
DRAM	32 GB/s	23 ms

These bounds ignore the CPU. When the data come from a disk the CPU cost hardly matters, since the data arrive slowly anyway; when they are already in DRAM it dominates. Real programs do not reach these bounds, but should get as close as possible.

Measured times. Table 7 collects the running times of the implementations discussed below, each run twice in a row on the same machine. All of them are far from the DRAM bound: even the fastest takes 0.58 s, about 25 times the 23 ms of the estimate.

Implementation	First run	Second run
awk	2.9 s	2.5 s
Python	4.3 s	4.0 s
C++, `-O0`	1.9 s	1.6 s
C++, `-O3`	0.9 s	0.6 s
C++, `-O3`, `mmap`	0.9 s	0.58 s

Table 7: Running times of the sum query at SF = 1.

3.2 Interpreted and compiled code

awk and Python. In awk the query is one line, with | as field separator:

```
awk -F'|' '{ s += $5 } END { print s }' lineitem.tbl
```

In Python it is a loop over the lines of the file:

```
s = 0
with open(sys.argv[1]) as f:
    for line in f:
        s = s + float(line.split('|')[4])
print(s)
```

awk is a small, domain-specific language for line-oriented text processing (logs, CSV), handy for a one-off check of the data on the command line; Python is a high-level, general-purpose language for automation, glue code and scripting. Both are slow here (2.5 s and 4 s warm) because both are interpreted.

Interpreted versus compiled. An **interpreted** language is executed by an interpreter, which pays at run time for garbage collection, dynamic type checking, function lookups and similar work; Python in particular is notoriously hard to optimise for performance. In a **compiled** language most checks and optimisations happen once, at compile time, and at run time only the generated, optimised machine code runs. Static types, direct control of memory and zero-cost abstractions help the compiler further; C++ is a compiled, statically typed language.

The C++ version. The C++ program reads the file line by line and parses the fifth field by hand, without a generic conversion function:

```
while (getline(in, line)) {
  auto last = line.data(); unsigned column = 0;
  for (char& c : line)
    if (c == '|') {
      if (++column == 5) {                // c ends the 5th field
        unsigned v = 0;
        for (auto it = last; it != &c; ++it) v = 10*v + (*it - '0');
        sum += v; break;
      } else last = &c + 1;               // start of the next field
    }
}
```

The inner loop converts the digits between the fourth and the fifth separator into an integer, $v \leftarrow 10v + d$ for each digit d. Compiled without optimisations (`-O0`) it already beats awk; with `-O3`, the most aggressive optimisation level, the same source runs twice as fast (table 7), because the compiler rewrites the hot loop. The machine code it generates can be inspected, e.g. with the online Compiler Explorer (godbolt).

Counting events. The profiler `perf` reads the **hardware performance counters** of the CPU: cycles, instructions executed, branches, cache loads and misses. Table 8 compares the C++ version (`-O0`) and the Python one. The system time, spent in the kernel reading the file, is the same small amount for both; the difference is all user time. Python executes about four times as many instructions for the same work, which is the cost of interpretation, and its higher number of instructions per cycle (IPC) does not compensate.

	C++ (-O0)	Python
elapsed time	1.91 s	3.91 s
user / system time	1.79 / 0.12 s	3.80 / 0.11 s
cycles	$8.4 \cdot 10^9$	$17.0 \cdot 10^9$
instructions	$10.0 \cdot 10^9$	$42.0 \cdot 10^9$
instructions per cycle	1.20	2.48
branches	$2.2 \cdot 10^9$	$9.7 \cdot 10^9$
L1 data-cache loads	$5.2 \cdot 10^9$	$11.0 \cdot 10^9$

Table 8: Hardware counters of the sum query, C++ and Python.

Profiling. To see *where* a program spends its time, a sampling profiler (`perf record`, then `perf report`) interrupts it at regular intervals and attributes each sample to the function running at that moment, including the kernel functions executed on its behalf; the report is a call tree in which each function has a *self* share (samples in the function itself) and a *children* share (including the functions it calls). For Python, the specialised profiler `scalene` separates time in Python code, in native code and in the system (often waiting for I/O). The profiles of the C++ versions are discussed with file reading below.

3.3 Reading a file: page cache, mmap and direct I/O

Cold and warm runs. Every implementation is faster the second time (table 7). The reason is the **page cache**: the operating system keeps recently read file data, of all processes, in otherwise unused DRAM. In the first (**cold**) run the file comes from the disk; in the second (**warm**) run it is already in DRAM. Several tools show it.

- `time` reports the elapsed (*real*) time and the CPU time spent in the program (*user*) and in the kernel on its behalf (*sys*). For awk, real/user/sys are 2.91/2.47/0.43 s cold and 2.51/2.37/0.14 s warm: the gain is mostly system time.
- `free` shows the memory in use, free, and used as buffers and page cache (*buff/cache*); after the cold run buff/cache has grown by exactly the size of `lineitem`. To measure a cold run again, the page cache must be dropped.
- `vmstat` and `iostat` show clear disk activity during a cold run and almost none during a warm one, which is mostly CPU time.
- `strace` counts the system calls: the Python version issues 92 780 `read` calls in both runs (about 725 MB/92 780 $\approx$ 8 KB each), but they take more time in the cold run (0.064 s against 0.029 s), when they block waiting for the disk; in the warm run they are page cache hits.
- `perf` counts block-I/O requests to the disk: 5826 issued in the cold run of the C++ version, none in the warm run.

Buffered I/O. With traditional (**buffered**) I/O, the call `read(fd, buf, n)` asks the kernel to copy n bytes of the file into the buffer `buf` in the address space of the process (fig. 16). If the data are in the page cache the kernel copies them and returns at once. Otherwise it programs a DMA transfer from the storage device into the page cache and blocks the calling thread; when the device signals completion (an interrupt), the kernel copies the data from the page cache into `buf` and the call returns. The costs of this path are:

- the lookup in the page cache;
- **read-ahead**: when the kernel has to go to the disk it reads more than requested (more than the 4 KB asked for), anticipating a sequential scan;

- the **double copy**: device to page cache (by DMA), then page cache to the application buffer (by the CPU);
- the system-call overhead of every `read`, which adds up for a sequential scan made of many small reads.

Writing is the reverse: `write` copies the data into the page cache and returns, and the kernel writes them back to the disk later.

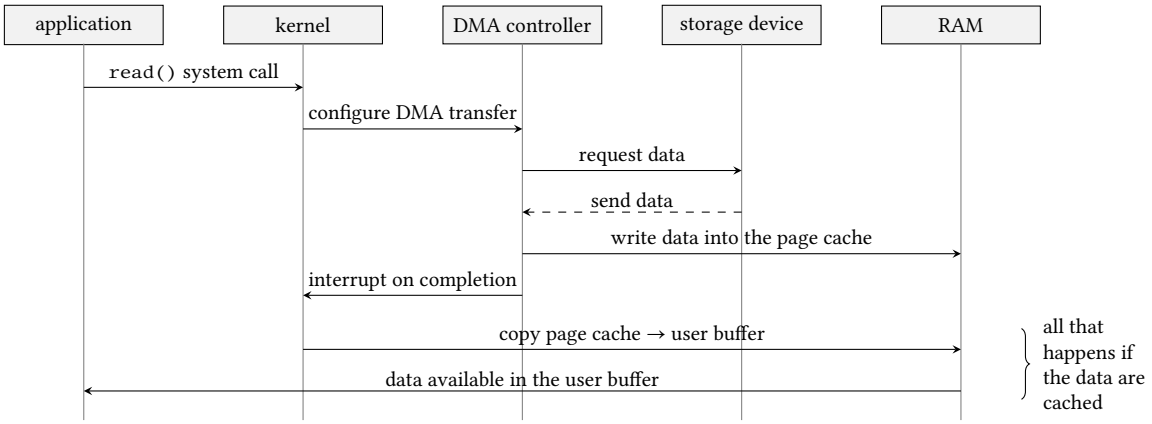


Figure 16: Buffered read of a file.

Where the time goes with read. In the profile of the C++ version (`-O3`, warm run) about 70% of the samples fall in the parsing loop itself and 25% in reading lines, of which 14.5% inside the `read` system call and 5.5% in the copy from the page cache to the buffer. In the cold run `read` takes about 26% of the samples, more than half of them spent on read-ahead from the disk.

Memory-mapped files. The system call `mmap` maps a file into the virtual address space of the process (section 2.4): the pages of the page cache become directly visible to the program, which reads the file as ordinary memory. There is no copy into a buffer and, after the one `mmap` call, no system call per read. For the whole file:

```
data = mmap(nullptr, size, PROT_READ, MAP_SHARED, fd, 0);
```

returns a pointer to `size` bytes of the file opened as `fd`. Pages are loaded lazily (fig. 17): `mmap` only reserves a range of virtual addresses; the first access to a page finds no page table entry and raises a **page fault**; the kernel brings the file page into a physical page of the page cache (if it is not there already), writes the page table entry, and the access resumes, its translation then cached in the TLB. Later accesses to the same page do not involve the kernel. A fault that must read the disk is a **major** fault, one that only maps a page already in the page cache a **minor** fault.

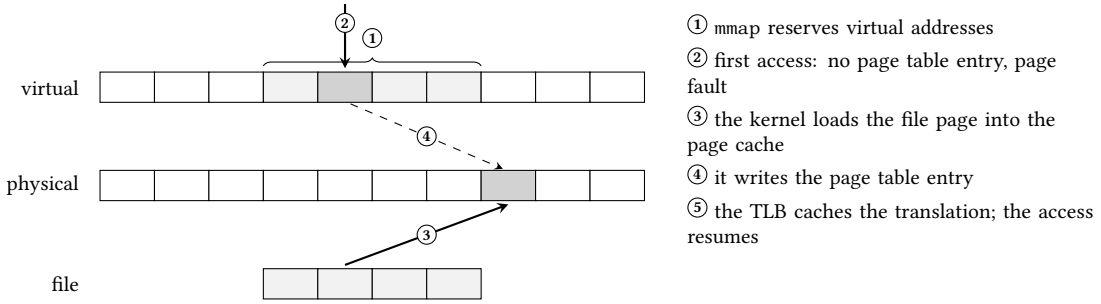


Figure 17: First access to a memory-mapped file.

read versus mmap, measured. Table 9 compares the two C++ versions (`-O3`) under `perf`. The `mmap` version trades `read` calls for about 11 700 page faults, almost all minor; it spends less time in the kernel, since nothing is copied, and is faster in both runs. Its profile shows the same shift: in the cold run about 28% of the samples handle page faults (with read-ahead of the file), in the warm run about 4%, and most of the time is spent parsing (about 41% in the function searching for the end of the line).

Version	Run	Elapsed	User	System	Page faults (major)	Block requests
`read`	cold	1.26 s	0.91 s	0.31 s	128 (4)	5826
`read`	warm	0.98 s	0.86 s	0.12 s	127 (0)	1
`mmap`	cold	1.03 s	0.73 s	0.22 s	11 719 (6)	5807
`mmap`	warm	0.76 s	0.70 s	0.06 s	11 719 (0)	0

Table 9: Reading the file with `read` and with `mmap`.

Direct I/O. **Direct I/O** transfers data between the storage device and the application buffer, bypassing the page cache. It avoids the double copy, reduces the CPU overhead and prevents *page cache pollution* (data read once evicting useful cached data), but the application must manage its own buffers, aligned to the block size (e.g. 4 KB), and gets no read-ahead and no caching from the kernel. Table 10 compares the three ways of reading a file.

	Buffered I/O	`mmap`	Direct I/O
data path	disk → page cache → buffer	disk → page cache, mapped	disk → buffer
copies	two	one (on the page fault)	one
system calls	every `read`/`write`	one `mmap`, then page faults on first access	every `read`/`write`
buffers	managed by the OS	managed by the OS	by the application, aligned (e.g. 4 KB)
read-ahead	yes	yes	no
strengths	simple, automatic caching	zero copy after the fault, efficient random access	highest throughput, no cache pollution
weaknesses	double copy, cache pollution on large or random I/O		application manages caching and alignment
typical use	general file access, small sequential reads	databases, large files, random access	high-performance databases, logging, large sequential I/O

Table 10: Buffered I/O, memory mapping and direct I/O.

4 Parallelism: SIMD and multithreading

The fastest single-threaded version of the sum query of section 3 reads the file at about 1.2 GB/s, far below the 32 GB/s of DRAM. This section explains why one thread cannot do much better, and how data-level parallelism (SIMD) and task-level parallelism (multithreading) close most of the gap; it ends with how parallel performance is measured.

4.1 Why one thread is not enough

Memory requests in flight. A thread typically issues one or a few memory loads, waits until the data arrive, and then continues: only a handful of requests are in flight at any time, and the memory channels stay mostly idle. With a DRAM latency of 60 ns, one cache line (64 B) per request and about 10 outstanding requests per thread (counting those that ILP overlaps), the read throughput of one thread is at most

$$\frac{10 \times 64\,\mathrm{B}}{60\,\mathrm{ns}} \approx 10.7\,\mathrm{GB/s},$$

a third of the 32 GB/s of the example machine. Read the other way round, sustaining a bandwidth B at latency L needs $B \cdot L/64\,\mathrm{B}$ requests in flight: $32\,\mathrm{GB/s} \times 60\,\mathrm{ns}/64\,\mathrm{B} = 30$ here, and hundreds for the memory systems of servers. One thread cannot keep that many loads outstanding; several threads can.

Classes of parallelism. Applications offer two kinds of parallelism: **data-level parallelism** (DLP), many data items processed at the same time, and **task-level parallelism** (TLP), different tasks operating independently and in parallel. Hardware exploits them in four ways:

- **instruction-level parallelism** (ILP) exploits DLP within one instruction stream, by pipelining and speculative execution;
- **vector architectures** (SIMD units, GPUs) exploit DLP by applying the same instruction to a collection of data;
- **thread-level parallelism** exploits DLP and/or TLP with hardware threads that work in parallel (several cores);
- **request-level parallelism** exploits DLP and TLP with largely decoupled tasks specified by the programmer, at large scale (distributed computing, data centres).

The abbreviation TLP thus denotes both the task-level parallelism of an application and the thread-level parallelism of the hardware that runs it; section 2 uses it in the second sense.

4.2 Data-level parallelism: SIMD

The SIMD model. Most modern processors contain a **SIMD unit** (or **vector unit**; **vector processors** are whole systems built on the same idea): a single instruction stream operates on multiple data streams. Its operands are **SIMD registers** (**vectors**) holding several values, the **lanes**, and one instruction applies the same operation to all lanes at once (fig. 18). An AVX-512 processor, for instance, has 32 registers of 512 bits (`zmm0` to `zmm31`), each usable as 64 8-bit, 32 16-bit, 16 32-bit or 8 64-bit integers, or as 16 32-bit or 8 64-bit floating-point numbers; fig. 3 shows how register widths grew.

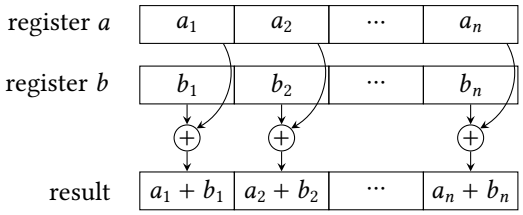


Figure 18: Lane-wise addition of two SIMD registers.

SIMD instructions make independence explicit: the lanes of one vector instruction never depend on each other, so the hardware checks *data hazards* (an instruction needing the result of an earlier one) only between vector instructions, not inside them. Data may have to be aligned in memory to the width of the SIMD unit, otherwise the compiler may generate scalar instructions for code that could be vectorised. With n lanes the promised speed-up is n-fold, but it is not reached in practice: vector code spends extra instructions on trivial things, such as converting data and moving them to the right position in the register, or emulating branches with conditional moves.

Three ways to write SIMD code.

- **Auto-vectorisation**: the compiler detects loops it can vectorise. The code stays platform independent, but it is worth checking in the generated assembly (`objdump`, godbolt) that it happened.

- **Compiler attributes**: vector types declared with `__attribute__((vector_size(...)))` state the intention explicitly. The code stays platform independent: where SIMD is not available the compiler generates scalar code.
- **Intrinsics**: functions that the compiler maps one to one onto SIMD instructions. They give explicit, instruction-level control without writing assembly, and the compiler still allocates the registers, better than hand-written assembly would. The price: the code is tied to one architecture (or needs a scalar fallback), is harder to read, and careless glue code (casts, conversions) can make it inefficient.

Rule of thumb: try auto-vectorisation and check its output; use intrinsics only in the few hot loops where they pay off.

Example 4.1 (Auto-vectorisation)**.** With three arrays of 256 `int`s, the loop

```
for (i = 0; i < 256; i++) c[i] = a[i] + b[i];
```

compiled with `-O3` becomes

```
.L3: movdqa (%rdx,%rax), %xmm0       # load 16 bytes (4 ints) of a
     paddd  (%rcx,%rax), %xmm0       # add 4 ints of b, lane by lane
     movaps %xmm0, 2048(%rsp,%rax)   # store the 4 sums into c
     addq   $16, %rax                # byte offset += 16
     cmpq   $1024, %rax              # end: 256 ints = 1024 bytes
     jne    .L3
```

where `%rax` holds the byte offset, `%rdx` and `%rcx` point to `a` and `b`, and `%xmm0` is a 128-bit SIMD register. `movdqa` moves an aligned packed integer value, `paddd` adds packed 32-bit integers (`d` for doubleword), `movaps` moves an aligned packed single-precision value (here simply 16 bytes). Each iteration handles four elements, so the loop runs $1024/16 = 64$ times instead of 256.

Example 4.2 (Vector types)**.**

```
typedef int v4si __attribute__((vector_size(16)));
union int_vec { int val[4]; v4si vec; };
int_vec a, b, c;  // a.val = {1,2,3,4}, b.val = {100,200,300,400}
c.vec = a.vec + b.vec;                          // one SIMD addition
```

The union gives access to the same 16 bytes as four scalars or as one vector. Even without optimisation (`-O0`) the addition becomes two SIMD loads (`movdqa`), one `paddd` and one store (`movaps`); the scalar values are first written to memory one by one (`movl`), i.e. data move between scalar and SIMD form through memory.

Intrinsics. Their names follow the pattern `_<size>_<op>_<suffix>`: the size is `mm` for 128 bits, `mm256` and `mm512` for 256 and 512 bits; the operation is `add`, `sub`, `mul`, …; the suffix gives the lane type, e.g. `ps` (32-bit float), `pd` (64-bit double), `epi32` (signed 32-bit integer), `epu16` (unsigned 16-bit integer). The types `__m128i`, `__m256i`, `__m512i` are integer registers of the three sizes. The addition of the previous example reads

```
__m128i x = _mm_loadu_si128((__m128i*) a); // load 16 B (vmovdqu)
__m128i y = _mm_loadu_si128((__m128i*) b);
x = _mm_add_epi32(x, y);                   // 4 int adds (vpaddd)
_mm_storeu_si128((__m128i*) c, x);         // store 16 B
```

The Intel Intrinsics Guide gives, for each operation, the intrinsic, the instruction, and its latency and throughput.

Masks. From AVX-512 on, almost every operation can take a bit **mask** k (type `__mmask16` for 16 lanes of 32 bits) and act only on the lanes whose bit is 1: conditional execution of individual lanes, without branches (fig. 19).

- **Zero masking** (`maskz` in the intrinsic's name): the operation is applied to the selected lanes, the others become 0.
- **Merge masking** (`mask`): the selected lanes get the result, the others keep the value of an extra source vector `src`, so a new result is merged into a previous one. A **blend** takes each lane from a second vector where k is 1 and from the first elsewhere.
- **Compress** packs the selected lanes, in order, into the first lanes (the rest set to 0); **expand** does the inverse, spreading the first lanes over the selected positions. Variants write to or read from memory directly. Compress is how a vectorised selection writes out only the qualifying values.

The intrinsics are listed in appendix C.

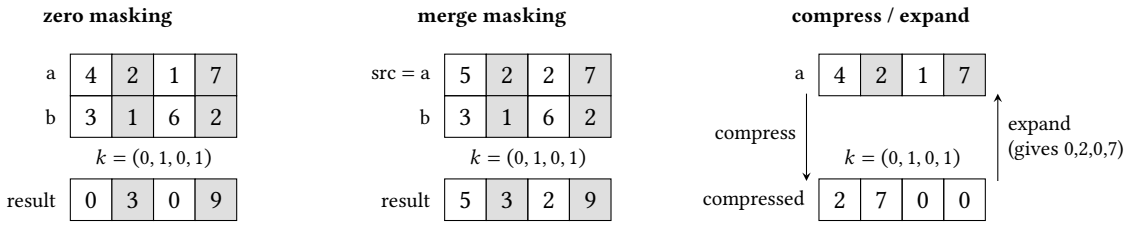


Figure 19: Zero masking, merge masking, compress and expand.

Vectorising the sum query. `-O3` does not vectorise the parsing loop of section 3.2. Asked to report its decisions (appendix C), the compiler lists the reasons: control flow in the loop, iteration count not known in advance, several exits. Branches and `breaks`, dependencies between iterations, loops whose length is unknown on entry and the lack of predication all prevent auto-vectorisation, and character parsing is not an idiom the compiler recognises. Yet searching for the end of the line takes a large share of the time (about 41% of the samples in section 3.3), and it can be done several bytes at a time by hand.

SIMD within a register. Before using vector instructions, a plain 64-bit register can already process 8 bytes at once: **SWAR** (SIMD within a register) performs the same operation on all bytes of a machine word by careful masking and shifting. Two building blocks:

- *Broadcast*: `b * 0x0101010101010101` puts the byte b into all 8 bytes, since $b \sum_{i=0}^{7} 2^{8i} = \sum_{i=0}^{7} b\, 2^{8i}$ and $b < 2^8$ produces no carries.
- *Zero bytes*: `z = (v - 0x0101010101010101) & ~v & 0x8080808080808080` marks with `0x80` the bytes of v that are `0x00`. For one byte x, the top bit of $(x-1)\ \&\ {\sim}x$ is set only if $x = 0$: then $x - 1 = \texttt{0xFF} \pmod{256}$ and ${\sim}x = \texttt{0xFF}$; if $1 \le x \le \texttt{0x80}$, then $x - 1 < \texttt{0x80}$ has top bit 0; if $x > \texttt{0x80}$, then ${\sim}x < \texttt{0x80}$ has top bit 0.

Example 4.3 (Finding zero bytes)**.** For v = [`41 00 7F 80 00 10 FF 01`] (byte 7 on the left, byte 0 on the right), with A = `v - 0x01..01` and B = `~v`:

	7	6	5	4	3	2	1	0
v	41	00	7F	80	00	10	FF	01
A	3F	FF	7E	7E	FF	0F	FE	00
B	BE	FF	80	7F	FF	EF	00	FE
A & B	3E	FF	00	7E	FF	0F	00	00
z	00	80	00	00	80	00	00	00

The two zero bytes (6 and 3) are marked. The subtraction works on the whole word, so a zero byte borrows 1 from the byte above it: byte 4 becomes $80 - 1 - 1 = 7E$ and byte 7 becomes $41 - 1 - 1 = 3F$.

Remark. Because of that borrow, a byte `0x01` directly above a zero byte is marked too ($01 - 1 - 1 = FF$). The test is exact for the lowest zero byte and for deciding whether the word contains a zero byte at all ($z \neq 0$), which is what a search for the first match needs.

Finding newlines with SWAR. The search for `'\n'` (`0x0A`) uses a variant that avoids borrows and carries altogether:

```
uint64_t block = *reinterpret_cast<const uint64_t*>(iter);   // next 8 bytes
constexpr uint64_t high    = 0x8080808080808080ull;
constexpr uint64_t low     = 0x7F7F7F7F7F7F7F7Full;
constexpr uint64_t pattern = 0x0A0A0A0A0A0A0A0Aull;          // '\n' broadcast
uint64_t ascii_mask = (~block) & high;
uint64_t matches0A  = ~((((block & low) ^ pattern) + low) & high);
uint64_t matches    = matches0A & ascii_mask;
```

Byte by byte, for a byte x of `block`:

1. `x & 0x7F` clears the top bit, so each byte is at most `0x7F`;
2. `^ 0x0A` gives 0 exactly when the low 7 bits of x equal `0x0A`;
3. `+ 0x7F` sets the top bit exactly when the result of step 2 is not 0, and never carries into the next byte, since $7F + 7F = FE < 100$;
4. `& high`, then ~: the top bit is now set exactly when the low 7 bits equal `0x0A` (the other bits are 1 and are discarded next);
5. `& ascii_mask` keeps only top bits, and only for bytes whose own top bit was 0, which excludes bytes such as `0x8A`.

So `matches` holds `0x80` in exactly the bytes equal to `'\n'`, and the same code with another pattern (e.g. `'|'`) finds any delimiter 8 bytes at a time. On x86 the first byte in memory is the lowest byte of the word, so the position of the first match is the number of trailing zero bits of `matches` divided by 8.

Example 4.4 (Three bytes through the newline test)**.**

byte x	`0A` (`'\n'`)	`41` (`'A'`)	`8A`
`x & 7F`	0A	41	0A
`^ 0A`	00	4B	00
`+ 7F`	7F	CA	7F
`& 80`, then ~	FF	7F	FF
`ascii_mask` (`~x & 80`)	80	80	00
`matches`	80	00	00

Only the newline is marked; `0x8A` passes the first four steps and is removed by the ASCII mask.

With SWAR the sum query takes 0.65 s cold and 0.38 s warm, against 0.6 s warm before.

Finding newlines with AVX2. With real SIMD instructions the same search covers 32 bytes per step:

```
auto block = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(iter));
uint64_t matches = _mm256_movemask_epi8(
                     _mm256_cmpeq_epi8(block, _mm256_set1_epi8(c)));
```

`_mm256_loadu_si256` loads 32 bytes, `_mm256_set1_epi8(c)` broadcasts the character `c` into all 32 byte lanes, `_mm256_cmpeq_epi8` compares lane by lane (`0xFF` where equal, 0 elsewhere), and `_mm256_movemask_epi8` collects the top bit of each lane into an integer: bit i is 1 exactly when byte i equals `c`. The hardware lanes replace the arithmetic tricks of SWAR. Here this version takes 0.7 s cold and 0.42 s warm, no faster than SWAR.

4.3 Task-level parallelism

Using several cores requires four design decisions: how to partition the work, how large the tasks are, how they are scheduled, and how correctness is ensured.

Work partitioning. The problem is divided into independent sub-problems (tasks) that threads can execute concurrently; this is also called **domain decomposition**. To sum an array, for instance, each thread gets a chunk of indices, computes a local sum, and the local sums are merged at the end (fig. 20). Equal chunks are not always enough: the length may not be divisible by the number of threads (uneven chunks), some chunks may cost more than others, e.g. when zeros need no work (unbalanced load), and data such as a sparse matrix with rows of variable length do not split into equal pieces at all (irregular partitioning). Partitioning should equalise the *work*, not just the data size.

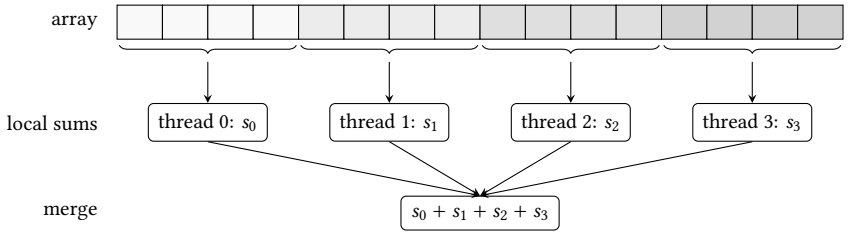


Figure 20: Parallel sum of an array with four threads.

Task granularity. The **granularity** of a task is its size or cost. Fine-grained tasks balance the load better but cost more scheduling and synchronisation overhead; coarse-grained tasks cost less overhead but may leave threads idle. In OpenMP the chunk size is a parameter of the parallel loop:

```
#pragma omp parallel for schedule(static, CHUNK) reduction(+:sum)
for (int i = 0; i < N; ++i) { sum += A[i] * B[i]; }
```

The pragma splits the iterations among threads in chunks of `CHUNK` iterations, and `reduction(+:sum)` gives each thread a private partial sum that is added up at the end. With `CHUNK = 1` scheduling and synchronisation overhead dominate; with `CHUNK = N/threads` each thread gets one huge chunk and threads finish at different times (load imbalance); an intermediate value such as 1000 balances the two.

Scheduling. The **schedule** is the runtime policy that assigns tasks to threads. **Static** scheduling fixes the chunks of each thread in advance; **dynamic** scheduling lets each thread take the next available chunk at run time; **guided** scheduling and **work stealing** adapt the load balance as the run proceeds. Static scheduling is cheapest when all iterations cost the same; when the cost varies, e.g. a loop whose iteration i busy-waits i mod 1000 units, dynamic scheduling keeps all threads busy (fig. 21).

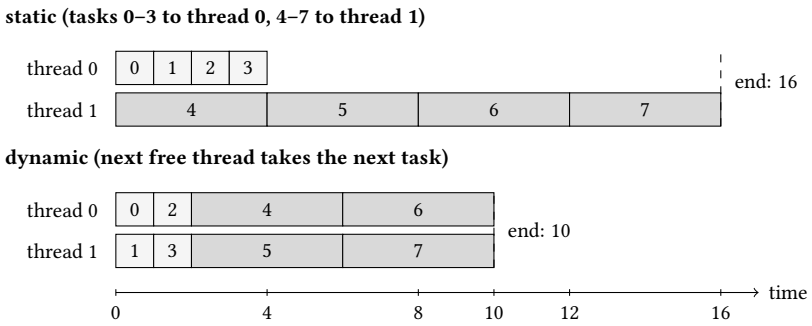


Figure 21: Static and dynamic scheduling of eight tasks of increasing cost on two threads (illustrative).

Correctness. The order in which threads read and write shared data is non-deterministic, so **synchronisation** is needed wherever the order matters; the reduction above avoids it by giving each thread its own partial sum.

Processes, threads and fibers. The operating system offers several constructs for concurrency and parallelism (table 11).

Construct	What it is	Memory, scheduling
process	instance of a program, isolated from the other processes: a virtual computer	own private memory; scheduled by the kernel
thread	locus of control inside a running program: a virtual processor	shares all the memory of its process; scheduled by the kernel
user-level thread	behaves like a thread, implemented in user space	invisible to the kernel; scheduled preemptively or cooperatively
fiber	lightweight thread of execution with cooperative multitasking	yields itself to let another fiber run

Table 11: Constructs for concurrency and parallelism.

The parallel sum query. The file is cut into one chunk per hardware thread; the main thread starts a thread for every chunk but the first, processes chunk 0 itself, and adds up the partial sums:

```
unsigned chunks = thread::hardware_concurrency();
vector<thread> threads;
for (unsigned index = 1; index < chunks; ++index)
  threads.push_back(thread(sumChunk(index)));
sumChunk(0);
```

On 16 threads it takes 0.15 s cold and 0.04 s warm. Table 12 summarises the warm times of all versions with the resulting bandwidth (725 MB divided by the time). Implementation matters a lot: the parallel version is two orders of magnitude faster than Python, and even a desktop machine processes several GB/s. With 18 GB/s everything is saturated except DRAM, whose 32 GB/s remain the bound.

Version	Warm time	Bandwidth
awk	2.57 s	282 MB/s
Python	4.05 s	179 MB/s
C++ (-O3)	0.62 s	1208 MB/s
C++ with `mmap`	0.58 s	1250 MB/s
C++ with SWAR	0.38 s	1907 MB/s
C++ parallel (16 threads)	0.04 s	18 125 MB/s

Table 12: The sum query at SF = 1, all versions (warm runs).

4.4 Measuring parallel performance

Scalability. *Scalability* is an overloaded term, used for instance for how well a server copes with more clients. With $RT(N, X)$ the running time of a problem of size X on N resources, three notions are distinguished:

- **speed-up**: the same problem on more resources (e.g. cores), ideally linear in N;
- **scale-up**: an N times larger problem on N times the resources, ideally constant (equal to 1);
- **scale-out**: an N times larger problem on N times as many machines, ideally constant like the scale-up.

$$SpeedUp(N) = \frac{RT(1,X)}{RT(N,X)},$$

$$ScaleUp(N) = \frac{RT(1,X)}{RT(N,NX)}, \qquad ScaleOut(N) = \frac{RT(1,X)}{RT(N,NX)} \ (N \text{ machines}).$$

Speed-up. With T_1 the sequential time and T_p the time on p CPUs, the **speed-up** is $S_p = T_1/T_p$. It is **linear** if $S_p = p$, **sub-linear** if $S_p < p$ (a performance loss), and **super-linear** if $S_p > p$, which usually signals a poor sequential baseline. Speed-ups are sub-linear in practice because programs may not contain enough parallelism (some parts are inherently sequential), because parallelisation has overheads (mostly synchronisation), and because of architectural limits, above all memory contention when the program is memory bound.

Amdahl's law. Let a fraction p of the work be parallelisable (here p is a fraction, not a number of CPUs) and run it on n cores, while the fraction $1-p$ stays sequential. The time becomes $T_n = T_1\big((1-p)+p/n\big)$, so

$$S_n = \frac{T_1}{T_n} = \frac{1}{(1-p)+\frac{p}{n}} \xrightarrow{n\to\infty} \frac{1}{1-p}.$$

The sequential part bounds the speed-up whatever the number of cores: if 90% of the work is parallelised, the speed-up never exceeds $1/0.1 = 10$ (fig. 22). Hence every phase of an algorithm whose cost depends on the input size must be parallelised.

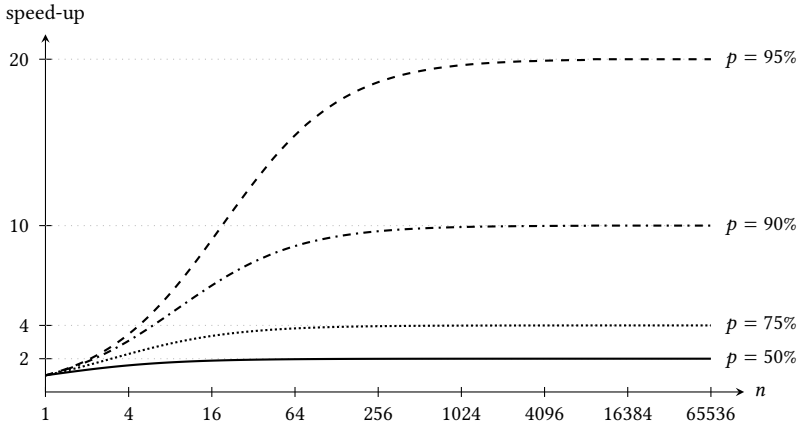


Figure 22: Amdahl's law for four parallel fractions.

Pitfalls. A **non-scalable algorithm** has to be rethought (in tree search, for instance, the order of exploration determines how much can run in parallel). **Load imbalance** is fixed by breaking the work into smaller tasks and scheduling them dynamically. **Task overhead** is fixed by a minimum task size per thread: tasks neither too small nor too large.

5 The relational model, basic SQL and query execution

The implementations of sections 3 and 4 make the programmer choose the file format, traversal order and parallelisation strategy. A database system separates the meaning of a query from these choices: the application requests a result, and the system decides how to compute it. This section introduces that separation, the relational operations expressed by SQL, and their translation into executable plans. SQL syntax is collected in appendix D.

5.1 Data models and data independence

Why a database system? A script operating directly on files depends on their format and organisation. Changing the encoding, restructuring the records or spreading growing data over several machines can break the script. Early **navigational database systems** had a similar problem: the programmer explicitly chose access paths and the order of operations, so changes to the database could require rewriting the program.

Definition 5.1 (Database management system)**.** A **database management system** (DBMS) is software for defining, creating, querying, updating and administering databases according to a **data model**, a collection of concepts describing the organisation of data.

Data models. The **relational model** describes data as tables. Other models include key/value (simple applications and caching), graph, document (JSON, XML or objects), wide-column or column-family, and array (vectors, matrices and tensors, used in science and machine learning). Hierarchical, network, semantic and entity-relationship models describe further ways of structuring data. The model determines the concepts visible to the application, rather than a particular file encoding.

Definition 5.2 (Relational terminology)**.** A **relation** is a set of **tuples** (rows), each with named **attributes** (columns) taking values from specified **domains** (allowed values, represented by data types in SQL). Its **degree** is its number of attributes, and its **cardinality** its number of tuples. A **schema** specifies this structure; an **instance** is the data currently stored.

For example, the schema `students(sid, name)` has degree 2; an instance containing three students has cardinality 3. Relations have neither duplicate tuples nor a tuple order. SQL normally uses **bags** (multisets), allowing duplicates; an output order must be requested explicitly.

The relational model separates three concerns: **structure** (relations independently of their physical representation), **integrity** (constraints that valid instances must satisfy), and **manipulation** (declarative operations on the relations).

Data independence. **Logical data independence** allows the logical schema to change while preserving the interface seen by applications, for example through external schemas or views. **Physical data independence** allows the storage layout, encoding, access structures or storage device to change without changing the logical schema (fig. 23). The DBMS can therefore adapt storage to its environment, data and workload without exposing those decisions to the application. An **external schema** is the application's view of the database; a **view** exposes a relation defined from other relations, so the same interface can be maintained when underlying tables change.

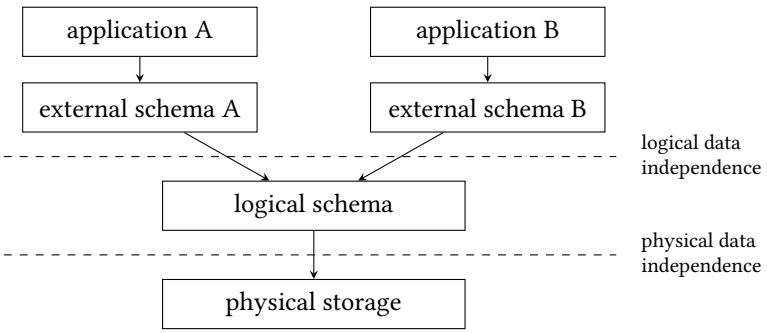


Figure 23: Logical and physical data independence.

Imperative and declarative processing. An **imperative** program specifies how to compute the answer: loops, algorithms, memory accesses and parallel tasks. A **declarative** query specifies which result is wanted; the engine chooses algorithms, data access and parallel execution. Computing an

average over rows satisfying a condition, for example, means explicitly filtering, accumulating a sum and counting in C++, but expressing only the condition and the average in SQL. Imperative code offers tight control; declarative processing delegates complex data operations to the engine, whose optimiser determines much of the performance.

SQL (Structured Query Language) grew from IBM's relational-language work in the 1970s, including SEQUEL for System R; it became an ANSI standard in 1986 and an ISO standard in 1987. Later revisions extended it with features such as recursion, window functions, XML, JSON and graph queries.

5.2 Relational operators and SQL semantics

SQL expresses the same relational operations that were implemented by text pipelines in section 1.3. Operators compose because each consumes relations and returns a relation (with SQL's bag semantics unless duplicates are eliminated).

Operator	Meaning	SQL counterpart
selection σ	retain rows satisfying a predicate	`WHERE`
projection π	retain or compute attributes	`SELECT`
rename ρ	name an attribute or relation	`AS`
union $\cup$	rows in either input	`UNION`
difference $-$	rows in the first input only	`EXCEPT`
intersection $\cap$	rows common to both inputs	`INTERSECT`
product $\times$	every pair of input rows	`CROSS JOIN`
join $\bowtie$	pairs satisfying a condition	`JOIN ... ON`, `USING`
grouping γ	one aggregate result per group	`GROUP BY`, `SUM`, `COUNT`, `AVG`
sorting τ, limiting	order rows, retain a prefix	`ORDER BY`, `LIMIT`

Table 13: Relational operators and SQL.

Sets and bags. Projection can create equal output rows even when the input rows differ. SQL retains these copies by default; `DISTINCT` removes them. Likewise, `UNION` removes duplicates, whereas `UNION ALL` keeps all occurrences from both inputs. `EXCEPT` and `INTERSECT` also eliminate duplicates by default. Set operations require compatible column counts and types. For one-column inputs $R = [1, 1, 2]$ and $S = [1, 3]$, their union is $\{1, 2, 3\}$, while their bag union contains three copies of 1, one of 2 and one of 3.

Join semantics. An **inner join** returns one combined row for every matching pair. A **left outer join** additionally retains every unmatched left row, filling the right attributes with `NULL`; right and full outer joins retain unmatched rows from the right or from both sides. `NULL` denotes a missing or unknown value. A **cross join** returns all $|R|\,|S|$ pairs without a condition. A **semi-join** retains left rows having at least one match, without copying the right attributes or multiplying the left row by its number of matches; an **anti-join** retains left rows having no match.

Example 5.3 (Join results)**.** Let `students(sid, name)` contain $(1, \text{Ana})$, $(2, \text{Ben})$, $(3, \text{Cleo})$, and `enrolled(sid, course)` contain $(1, \text{DB})$, $(1, \text{OS})$, $(2, \text{DB})$. Joining on equal `sid` gives:

Join	Result (showing names and courses where present)
inner	(Ana, DB), (Ana, OS), (Ben, DB)
left outer	the three inner rows and (Cleo, `NULL`)
semi	Ana, Ben
anti	Cleo
cross	$3 \times 3 = 9$ combined rows

Ana occurs twice in the inner join because she matches two enrolments, but once in the semi-join because only the existence of a match matters.

Subqueries and self-joins. A **subquery** is a query nested inside another query. `EXISTS` tests whether its result is nonempty; `NOT EXISTS` tests whether it is empty, expressing semi- and anti-joins. A **correlated subquery** refers to a row of the surrounding query, so the matching condition is evaluated for that row. `IN` tests membership in a subquery result; `NOT IN` is its SQL negation, with special care needed for `NULL` (appendix D). In `FROM`, a subquery acts as a **derived table**, whose output rows are the input of the surrounding query. Counting the rows of a grouped subquery after `HAVING` therefore counts qualifying groups, not their input rows.

A **self-join** uses the same relation in two roles, distinguished by aliases. To find unordered pairs of distinct entities sharing an attribute, join the two copies on that attribute and require the first identifier to be smaller than the second. This removes self-pairs and reversed pairs; `DISTINCT` is still needed if several shared values can generate the same pair.

Filtering, grouping and ordering. `WHERE` filters individual input rows before aggregation. `GROUP BY` partitions the remaining rows by equal grouping attributes, and aggregates compute one result per group. `HAVING` filters these group results. Output attributes must identify the group or be aggregated; ordering then arranges the output rows, and limiting keeps the requested prefix. Thus filtering rows and filtering groups answer different questions. `COUNT(*)` counts rows, `COUNT(x)` counts non-`NULL` values, and `SUM` and `AVG` ignore `NULL` inputs. If no non-`NULL` value remains, `AVG` returns `NULL`, rather than an average of zero.

Example 5.4 (An analytical query across three tables)**.** The TPC-H database of section 3.1 also has `customer` (customer key, market segment) and `orders` (order key, customer key, date and shipping priority). Each `lineitem` references an order and supplies, among other fields, an extended price, discount and shipping date. Let $d = 15$ March 1995. Retain BUILDING customers, their orders placed before d, and those orders' line items shipped after d. For each order compute

$$revenue = \sum_{\substack{\text{matching line items} \\ \text{ship date} > d}} extendedprice\,(1 - discount).$$

Group by order key, order date and shipping priority, sort by decreasing revenue and then increasing order date, and retain the first ten results. Figure 24 shows the operations; the complete SQL is in appendix D.

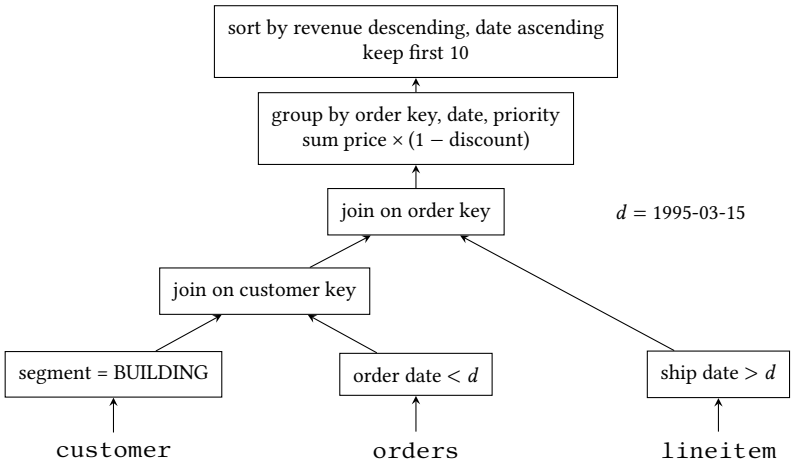


Figure 24: A logical plan for the TPC-H revenue query.

5.3 From SQL to physical execution

The system stack. A DBMS exposes a logical query language over a logical data model; below them, a physical compute layer executes queries on an instance, and a physical storage layer holds the data.

The same language and model can serve gigabytes on one machine or petabytes across a cluster. Data independence remains useful for other data shapes too, such as trees, arrays and graphs.

Parsing, binding and optimisation. The **parser** checks the query's syntax and builds an **abstract syntax tree**. The **binder** resolves table and attribute names, their identifiers and types using the **system catalog**, producing a **logical plan**: a tree of relational operations. The **optimizer** transforms it into a semantically equivalent **physical plan**, which chooses executable algorithms (fig. 25).

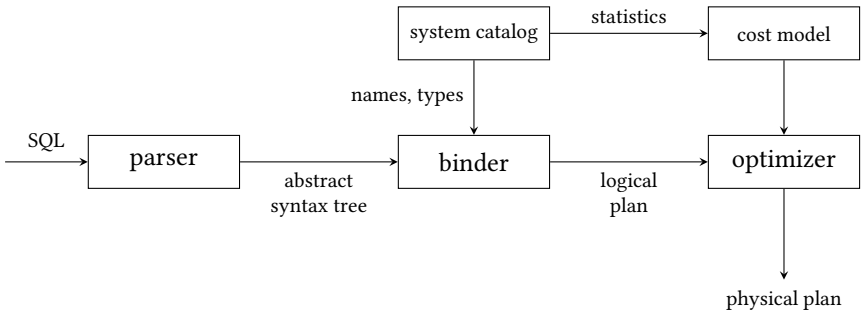


Figure 25: Query compilation.

Rule-based optimisation applies equivalence-preserving rewrites, such as moving a filter below an inner join when it depends on only one input. **Cost-based optimisation** compares alternatives using estimated table sizes, data distributions and **selectivities** (fractions of rows satisfying predicates). It can reorder joins and select physical operators. The logical operations describe the result independently of the system; PostgreSQL, MySQL and Spark SQL may implement them differently.

Physical operators. A scan can read a whole table or use an **index**, an auxiliary access structure for locating rows by a key. Aggregation can hash rows into groups, or process equal keys together after sorting. A logical join similarly admits several algorithms (table 14).

Algorithm	Mechanism	Favourable case
nested loop	For each outer row, search the inner input for matches; without an index, scan it.	Small outer input; an inner index with few matches per lookup.
hash join	Build a hash table on one input's join key; probe it with rows of the other input.	Equality join; unsorted inputs; build side fits in memory.
sort-merge join	Scan inputs ordered by the join key, combining matching key groups.	Inputs already sorted, or sorting is useful for later operations.

Table 14: Physical join algorithms.

For m outer rows, n inner rows and z output pairs, a plain nested loop costs $O(mn)$; with a suitable tree index the cost is roughly $O(m \log n + z)$. An in-memory equality hash join has expected cost $O(m + n + z)$. An equality sort-merge join on sorted inputs costs $O(m + n + z)$, plus sorting if needed. The output term matters: two large groups with the same key generate all pairs between them. Ordered inputs can also help range joins, whose cost depends on the predicate and number of matches.

Resources and parallel execution. Hash tables consume memory; sorting consumes CPU and may require extra I/O. If intermediate results exceed available DRAM, the engine may **spill** them to storage. Distributed joins may **shuffle** rows over the network to bring matching keys to the same machine, or **broadcast** a small input to the workers. Network traffic can then dominate. A query may create thousands or millions of tasks, with a queue and a dynamic, cost-aware scheduler controlling

concurrency and resource allocation. These choices connect query execution to the scheduling and memory limits of section 4.

Reading an execution plan. `EXPLAIN` shows the chosen plan and estimated statistics without running the query; `EXPLAIN ANALYZE` runs it and adds measured statistics. The tree is read from its input leaves towards the root. In PostgreSQL, each node reports estimated startup and total cost, output rows and row width; the measured fields report startup and completion time, output rows and execution count (`loops`). Cost units are model units, not milliseconds. A parent's cost includes its children, whereas its row count describes its own output. For repeated nodes, reported times and row counts are averages per execution, so `loops` is essential.

Example 5.5 (Estimates and measurements)**.** A query sums `bar` by `foo` in a table `test`, retaining rows with $100 <$ `id` < 200. Its plan uses an index scan followed by hash aggregation:

Node	Estimated cost	Estimated rows	Width (B)	Startup / end (ms)
index scan	0.29–10.27	99	8	0.009 / unavailable
hash aggregate	10.77–10.87	10	12	0.043 / 0.044

The aggregate produces 10 rows in one execution. Planning takes 0.244 ms and execution 0.073 ms. Estimated and observed cardinalities can be compared where both are present; costs and times cannot be compared numerically because their units differ. Parallel plans also report the workers planned and actually launched.

Same logic, different plans. Consider joining `Employees` and `Salaries` on employee identifiers, then joining `Departments` on department identifiers and retaining departments at location HQ. A traditional database with selective indexes can start from the qualifying departments and use indexed nested loops into employees and salaries. Spark with Iceberg can instead scan Parquet partitions, filter the departments, and use distributed hash joins with broadcasts or shuffles. These are possible plans for the same logical request: available indexes, storage layout, statistics, resources and parallelism determine the choice.

6 Advanced SQL: aggregation and query composition

SQL can combine several levels of aggregation, use a query's result inside another query, and evaluate a table expression for each input row. These constructs extend the operators of section 5; their syntax is collected in the same SQL reference, appendix D.

6.1 Multiple grouping levels and statistical aggregates

Running example. Let `items(id, score)` contain $(1, 4.0)$, $(2, 3.9)$ and $(3, 3.5)$. The relation `links(id, category, label)` associates items with categories:

id	category	label
1	P	C
3	P	B
2	Q	A
1	Q	C
2	R	B

The relation `categories(category)` contains P, Q, R and S; S has no links. Item identifiers and category names are unique in their respective tables, and each item–category pair occurs at most once in `links`. Each link references an existing item and category. Joining items to links and averaging scores by category gives 3.75 for P, 3.95 for Q and 3.90 for R. Category S is absent from this inner join. Grouping alone does not order the result.

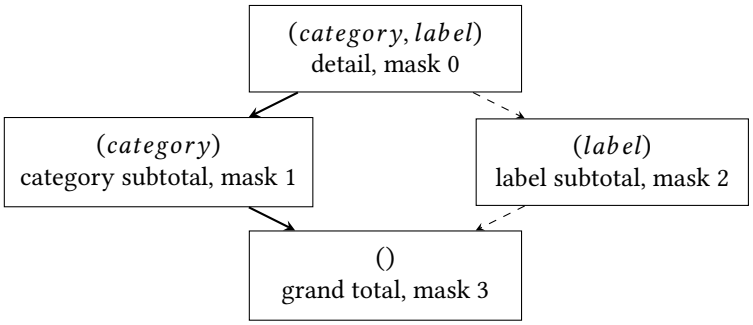
Grouping sets. Ordinary grouping computes one aggregation level. **Grouping sets** request several levels in the same query: grouping by $(category, label)$ gives detail groups, by $(category)$ gives category subtotals, and by the empty set () gives one grand total. Columns absent from a grouping set appear as NULL. The combined result has UNION ALL semantics: each grouping contributes its rows, without duplicate elimination.

Example 6.1 (Detail, subtotals and total)**.** Counting links at these three levels produces nine rows:

category	label	Count	Level
P	B	1	detail
P	C	1	detail
P	NULL	2	category subtotal
Q	A	1	detail
Q	C	1	detail
Q	NULL	2	category subtotal
R	B	1	detail
R	NULL	1	category subtotal
NULL	NULL	5	grand total

The equivalent formulation combines three grouped queries with UNION ALL, inserting NULL in the omitted columns. A grouping-sets implementation can share input processing across levels; the actual scan count depends on the physical plan.

Hierarchies and combinations. **ROLLUP** walks from detail through successive prefixes of an ordered list of n grouping expressions to the grand total, giving $n + 1$ sets. For category then label, these are $(category, label)$, $(category)$, (). Reversing the expressions changes the subtotal dimension. **CUBE** uses all 2^n subsets; with two dimensions it also includes $(label)$, giving totals A: 1, B: 2 and C: 2 in the example.



ROLLUP(category, label): solid path CUBE(category, label): all four nodes

Figure 26: Aggregation levels.

Two meanings of NULL. A missing grouping column and an actual missing input value can look identical. GROUPING(x) is 1 when x is omitted from the current grouping set and 0 when it is retained, even if its value is NULL. With several arguments the result is a bitmask, with the rightmost argument in the least significant bit. Thus GROUPING(category, label) is 0 for detail, 1 for a category subtotal, 2 for a label subtotal and 3 for the grand total (fig. 26). Some engines also call this operation GROUPING_ID.

Example 6.2 (An empty category)**.** Left-join categories to links, retain R and S, and roll up category then label, counting non-NULL link identifiers:

category	label	Count	Grouping mask
R	B	1	0
R	NULL	1	1
S	NULL	0	0
S	NULL	0	1
NULL	NULL	1	3

The two S rows have different meanings: one groups the missing label supplied by the outer join, the other summarises all labels. Their masks distinguish them. Counting the link identifier gives 0 for S, whereas counting rows would count its one padded outer-join row. Labels such as "subtotal" should therefore be selected using the grouping mask, rather than testing only for NULL.

Statistical summaries. Beyond sums, counts and extrema, SQL engines offer population and sample standard deviation, covariance, Pearson correlation, and the slope and intercept of a least-squares regression line. Population and sample versions use different normalisations; regression functions take the dependent variable y before the explanatory variable x. The reference lists the corresponding function names.

Streaming aggregates update a fixed-size state as rows arrive: for example, an average needs a sum and count, and a standard deviation can maintain a count, mean and accumulated squared deviations. They need no sorting and constant state *per group*; maintaining many groups still consumes memory proportional to their number.

Ordered-set aggregates receive a value ordering through WITHIN GROUP. A continuous percentile interpolates between neighbouring ordered values when necessary; a discrete percentile returns an input value, the first whose cumulative fraction reaches the requested percentile. At $p = 0.5$ these give continuous and discrete medians. Exact percentiles generally retain and sort values, unlike the fixed-size streaming summaries. MODE returns a most frequent value; an exact implementation needs frequencies or ordered values, and ties can have several valid answers.

Example 6.3 (Continuous and discrete medians)**.** Join the running example's items to links and aggregate by category:

Category	Ordered scores	Continuous median	Discrete median	Modal labels
P	3.5, 4.0	3.75	3.5	B or C
Q	3.9, 4.0	3.95	3.9	A or C
R	3.9	3.90	3.9	B

For P, interpolation gives $(3.5 + 4.0)/2 = 3.75$, while the first half of the ordered input is reached at 3.5. P and Q each have a tie for the most frequent label; which tied value MODE chooses depends on the engine. For large inputs, sampling or an approximate quantile algorithm can reduce the cost at the expense of exactness.

6.2 Nested queries, decorrelation and LATERAL

Result shape and scope. A nested query can supply a filter in WHERE or HAVING, a derived table in FROM, or a value in SELECT. A **scalar subquery** supplies one column and at most one row: no rows gives NULL, and more than one row is an error. An aggregate without grouping normally returns one row even on empty input, such as a zero count or a NULL average.

An **uncorrelated subquery** uses no outer-row attributes, so its result can be computed independently and reused. A correlated one depends on the surrounding row, as in section 5.2. These are logical dependencies, not an obligation to execute a particular loop: the optimizer may transform the query.

Choosing a predicate. A scalar comparison compares with a single derived value, for example the maximum linked item identifier. IN tests membership in a one-column result. More generally, ANY requires at least one comparison to be true, while ALL requires every comparison to be true; equality with ANY is IN. EXISTS only tests whether any row is returned and does not require correlation.

For the values 1, 2, 3, comparing 2 for equality with any value is true; asking whether 4 exceeds all values is true; whether 1 exceeds all is false; membership of 5 is false; existence of a value above 2 is true. On empty input, ANY and IN are false, ALL is true, and EXISTS is false. With NULL, a comparison can be unknown: ANY is unknown if none is true and at least one is unknown; ALL is unknown if none is false and at least one is unknown. Filters keep only true results.

Arrays and derived tables. An array constructor can collect all rows of a one-column subquery into one array value. For each category, collecting its labels in ascending order gives P: [B, C], Q: [A, C], R: [B], S: []. Without explicit ordering the element order is unspecified. A derived table instead exposes its rows and columns for further relational operations. It can pre-aggregate before a join, or let an outer count count the groups retained by an inner HAVING.

Decorrelation and cost. With m outer and n inner rows, repeatedly scanning the whole inner input costs $O(mn)$ in the worst case, quadratic when both sizes grow together. A suitable index can reduce each lookup; **decorrelation** rewrites the dependency into a join or another set-oriented operation. Whether the rewrite is chosen and helps depends on the query, indexes and plan.

For example, finding categories with no links is an anti-join. One formulation tests that no matching link exists; another left-joins links and keeps rows whose link identifier is NULL. The tested identifier must be non-NULL in a real matching row, so the test detects the join's padded row. Both return S in the example.

Lateral table expressions. **LATERAL** allows a FROM subquery to refer to preceding FROM items. For each outer row it supplies a table, possibly with several rows and columns, whose rows are joined to that outer row. A plain join is enough for a simple matching relation; lateral evaluation is useful when the per-row computation itself needs aggregation, ordering or a limit. A scalar subquery suffices when only one value is needed.

In the running example, a per-category count and average gives:

Category	Count	Average score	Highest-score item	Its score
S	0	NULL	NULL	NULL
R	1	3.90	2	3.9
P	2	3.75	1	4.0
Q	2	3.95	1	4.0

Two lateral aggregate subqueries can compute count and average separately, or one can compute both. Without grouping or HAVING, each aggregate subquery returns one row for every category, including S. The same summaries can also be obtained by an outer join and grouping, counting link identifiers rather than padded rows.

Top-k per group. To return the item identifier *and* score for each category's best item, a lateral subquery orders that category's items by decreasing score and keeps the first row. A left lateral join retains an empty category with NULL result columns; a cross lateral join drops it if the subquery produces no rows. A secondary ordering key resolves ties when one deterministic row is required. Computing only the maximum score does not retain the identity of the row that supplied it; a single scalar subquery supplies only one column, whereas the lateral result can expose both.

6.3 Strings, conditional expressions and sampling

Matching and transforming strings. `LIKE` matches a whole string against a pattern: `%` represents any sequence of characters, including the empty sequence, and `_` one character. `NOT LIKE` negates the test. Case-insensitive matching with `ILIKE` is an engine extension; `SIMILAR TO` is standard SQL pattern matching with regular-expression features, but is not implemented everywhere. POSIX regular expressions and their function syntax also vary across systems.

Substring extraction, upper/lower-case conversion and concatenation can be used in both output expressions and predicates. The standard concatenation operator is `||`; many engines also offer `CONCAT`. Regular-expression replacement can remove or replace matched text: removing a suffix matching `@.*$` extracts the user part of a login. A global replacement flag replaces every match; for example replacing each run matching `(ab)+` in `ababfooab` with `xy` gives `xyfooxy`.

Conditional values and conversion. `CASE` selects the result of the first true condition, in the written order. For thresholds "score at least 3.9" and then "at least 3.0", 3.9 takes the first branch; reversing them sends it to the broader 3.0 branch. If no condition is true, `ELSE` supplies the result, or `NULL` if `ELSE` is absent. Conditional expressions can appear inside an aggregate, for instance to sum a contribution of 1 for each true predicate and 0 otherwise.

`COALESCE` returns the first non-`NULL` argument, so it can replace an absent minimum label by "n/a". Under the ordering A < B < C, the minimum is the earliest label. `CAST` explicitly converts a value's type. A decimal type with precision 3 and scale 1 stores three decimal digits in total, one after the decimal point: the example averages 3.75 and 3.95 round to 3.8 and 4.0. Casting a `NULL` average leaves it `NULL`.

Sampling before computation. **TABLESAMPLE** replaces a base table by a random sample before filtering and aggregation. This can reduce downstream work, while statistics now describe sampled rather than complete data. **Bernoulli sampling** includes each row independently with probability p, so N input rows give an expected pN sample rows, with a varying actual count. **System sampling** selects whole pages or vectors with that probability: it is usually cheaper, but values clustered in those blocks are selected together. **Reservoir sampling** keeps a fixed-size random sample of k rows when at least k are available, and is useful for small samples.

Sampling and approximation trade accuracy for work; sampling can miss rare values, and a selective filter can leave very few sampled rows. A fixed seed supports repeatable comparisons on unchanged input, subject to the engine's execution rules. In DuckDB, guaranteed repeatability also requires single-threaded execution. Bernoulli sampling can still scan the entire table, so sampling does not guarantee less I/O.

7 Advanced SQL: common table expressions and window functions

This section completes section 6 with two ways of structuring analytical queries. Common table expressions name the steps of a query; their recursive form repeats a step until no new rows appear, which traverses hierarchies and graphs of unknown depth. Window functions compute values across rows, like aggregates, while keeping every row. The complete SQL is in appendix D.

7.1 Common table expressions

Naming intermediate results. Derived tables (section 6.2) are read from the inside out and become hard to follow when stacked. A **common table expression** (CTE), introduced in SQL:1999, instead names the intermediate result at the start of the statement:

```
WITH name (col_1, ..., col_k) AS (query)
SELECT ... FROM name ...;
```

The optional column list names the output columns of the inner query, like AS aliases: if `c(x, y)` is defined by `SELECT 1, 2`, then `SELECT x + y FROM c` returns 3. Within the statement the main query uses the name as if it were a table, and the name disappears when the statement ends. This distinguishes a CTE from the alternatives: a view (section 5.1) is stored in the catalog and visible to every query, and an explicit temporary table keeps its rows for the whole session.

For example, if `enrolled` also records a grade, a CTE can compute the average grade of each course, and the main query sorts these averages and keeps five. The result is that of the version with a derived table, but the query reads top-down, one named step after the other. Since `ORDER BY` is ascending by default, the five lowest averages are kept: the best courses when 1.0 is the best grade, as in Germany.

With `students` and `enrolled` of section 5.2, a CTE `maxCTE(maxID)` holding the largest identifier in `enrolled` contains the single row 2, and joining `students` with it on the identifier returns Ben; Cleo has the larger identifier 3 but no enrolment. A scalar subquery in `WHERE` gives the same result. The CTE names the value instead, so the query can reference it several times.

Several CTEs. One `WITH` clause may define several CTEs, separated by commas, and each may reference those defined before it. A query then reads as a pipeline: a first CTE computes a, a second computes b from a, a third computes c from b, and the main query reads c. An intermediate result used twice is written once.

Evaluation. An ordinary CTE is not recursive: its query is evaluated once, before the main query, and every reference sees the same rows.

7.2 Recursive common table expressions

Hierarchies and graphs as relations. Employees and their managers, folders and subfolders, and more generally the edges of a graph are stored as pairs of nodes: a hierarchy as one parent reference per row, a graph as an **edge relation**. One step along the edges is one join with this relation. A query with k joins therefore follows paths of at most k edges, a number fixed when the query is written, while the depth of a hierarchy is usually unknown. A **recursive CTE** repeats the step until no new rows appear; despite its name it is evaluated by iteration. Recursive CTEs make SQL Turing complete, as expressive as a general-purpose programming language.

Anchor, recursive member and working table. A recursive CTE combines two queries:

```
WITH RECURSIVE r AS (
    anchor_query        -- does not reference r: starting rows
  UNION ALL             -- or UNION
    recursive_query     -- references r: new rows from previous ones
)
SELECT * FROM r;
```

The **anchor member** supplies the starting rows, and the **recursive member** computes new rows from the previous ones. With `UNION ALL` the system evaluates them as follows, where the **working table** W holds the rows of the last iteration and R collects the result.

1. $W \leftarrow$ rows of the anchor member; $R \leftarrow W$.
2. While W is not empty: evaluate the recursive member with `r` standing for W, let W be its rows, and append them to R.
3. Return R.

The recursive member thus sees only the rows of the previous iteration, never the whole result. The loop stops at the first iteration that produces no rows, the **termination condition**, and the result contains the rows of all iterations.

Example 7.1 (A sequence)**.** The query

```
WITH RECURSIVE numbers(n) AS (
    SELECT 1                                    -- anchor
  UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 5       -- recursive member
)
SELECT * FROM numbers;
```

fills the working table as follows:

iteration i	0	1	2	3	4	5
W after iteration i	1	2	3	4	5	empty

The result is 1, 2, 3, 4, 5. The condition is tested on the old row n, before $n + 1$ is formed: in iteration 4 the row 4 passes and produces 5; in iteration 5 the row 5 fails, so W is empty. With `n <= 5` the row 5 passes too and the result is 1 to 6. Without a condition W is never empty, and the query runs until the integer overflows or a timeout stops it.

Traversing a hierarchy. The relation `animals(id, name, parent)` stores the tree of fig. 27: each row references the identifier of its parent, and the root's parent is NULL. To find the ancestors of turtle, the anchor selects the turtle row, and the recursive member joins `animals` with W, keeping the rows whose identifier equals the parent of a row of W. The iterations give turtle (parent 5), reptile (identifier 5, parent 1) and animal (identifier 1, parent NULL). Then W is empty, because a comparison with NULL is never true. The result includes the anchor row itself.

Swapping the roles in the join condition (a row whose *parent* equals the identifier of a row of W) walks downwards: from mammal, iteration 1 returns giraffe and tiger, and iteration 2 is empty. A tree has no cycles, so both walks end at the root or at the leaves and `UNION ALL` terminates. The relation itself can be defined in the same `WITH RECURSIVE` clause, as an ordinary CTE over a VALUES list (an inline constant table).

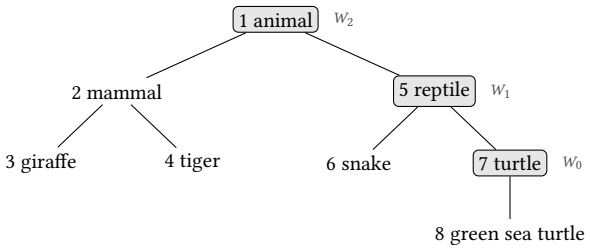


Figure 27: Ancestors of turtle in a hierarchy.

Cycles and UNION. In a graph with cycles `UNION ALL` need not terminate: a walk goes from Alice to Bob, back to Alice, to Bob again, and W is never empty. With UNION, duplicates are removed (section 5.2), also against the rows of earlier iterations. Writing δ for duplicate elimination and Q for the recursive member,

$$W_0 = \delta(\text{anchor}), \qquad W_{i+1} = \delta\big(Q(W_i)\big) \setminus (W_0 \cup \cdots \cup W_i),$$

and the loop stops at the first empty W_i. A row already in the result is neither added nor extended again.

Example 7.2 (Reachability in a graph)**.** The relation `friends(a, b)` stores each friendship of fig. 28 once, as a pair. The union of `friends` with its swapped copy (b, a), a CTE `friendship(name, friend)`, contains both directions. Starting from Alice, the recursive member returns the friends of the people in W:

i	$\delta(Q(W_{i-1}))$	W_i (new rows)
0	(anchor)	Alice
1	Bob, Carol	Bob, Carol
2	Alice, Dan, Grace, Chuck	Dan, Grace, Chuck
3	Bob, Anne, Carol, Chuck, Grace	Anne
4	Chuck, Dan	empty

The candidates already in the result (Alice in iteration 2; Bob, Carol, Chuck and Grace in iteration 3; Chuck and Dan in iteration 4) are dropped, and with them the cycles: in iteration 4 nothing new remains, so W is empty and the loop stops. The result has seven people; Eve and Adam are not connected to Alice.

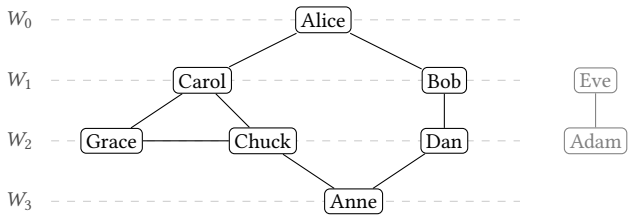


Figure 28: A friendship graph with cycles.

Termination and depth bounds. UNION terminates whenever the rows can take only finitely many values, for instance names taken from the database: every iteration before the last adds at least one new row, and there are only finitely many. A computed column breaks this guarantee. Recording the depth in a column d, set to 0 by the anchor and to $d + 1$ by the recursive member, makes (Alice, 0) and (Alice, 2) different rows. The walk from Alice to Bob and back is then no longer cut, and the iteration never stops, like the sequence without its condition. The recursive member needs a **depth bound**, a condition $d < k$. Hence UNION ALL for acyclic data, and UNION when there may be cycles, plus a bound when a column grows at every step.

7.3 Window functions

Aggregates that keep the rows. CTEs structure a query at the level of sets; **window functions**, part of SQL since SQL:2003, work at the level of rows. A window function computes, for each row, a value from a set of related rows, its **window**, and adds it to the row: like an aggregate, but without collapsing the rows into one per group. Running totals, moving averages, rankings, top-k queries, percentiles and comparisons with the previous row of a time series all follow this pattern.

Example 7.3 (Grouping versus windows)**.** The relation `sales(customer, day, amount)` contains the six sales in the first three columns below (days of January 2025). Grouping by customer and summing returns two rows, (A, 75) and (B, 30): the individual sales disappear. The window function `SUM(amount) OVER (PARTITION BY customer)` instead returns all six rows and adds 75 or 30 to each. With `ORDER BY day` inside OVER, the sum becomes the running total of each customer:

customer	day	amount	running total	3-row average	previous	change
A	1 Jan	10	10	10	NULL	NULL
A	2 Jan	20	30	15	10	10
A	3 Jan	15	45	15	20	−5
A	4 Jan	30	75	21.67	15	15
B	1 Jan	5	5	5	NULL	NULL
B	4 Jan	25	30	15	5	20

All windows partition by customer and order by day; the computation restarts for B. The last three columns are explained below.

Anatomy of a window. A window function has the form

```
f(args) OVER (PARTITION BY ... ORDER BY ... frame)
```

where f is an aggregate or a special window function, and OVER defines which rows f sees for the current row:

- PARTITION BY splits the rows into **partitions** of equal values, as GROUP BY forms groups; a window never leaves the partition of its row. Without it, all rows form one partition.
- ORDER BY orders the rows within each partition. Rows equal on the ORDER BY expressions are **peers**.
- The **frame** selects a part of the ordered partition, relative to the current row; it slides along as the current row moves (fig. 29).

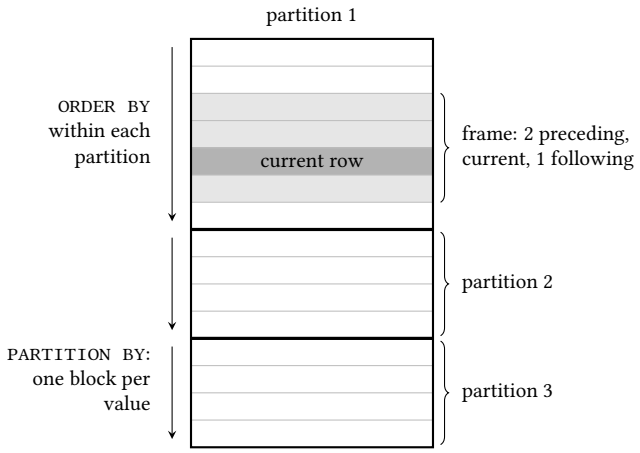


Figure 29: Partitions, order and frame.

Frames. A frame is written ROWS or RANGE BETWEEN *start* AND *end*. The unit ROWS counts rows: 2 PRECEDING means two rows before the current one. The unit RANGE uses the values of the ORDER BY expression: with value v in the current row, 3 PRECEDING means the rows with value at least $v - 3$ (fig. 30). The boundaries are UNBOUNDED PRECEDING (the start of the partition), n PRECEDING, CURRENT ROW, n FOLLOWING and UNBOUNDED FOLLOWING (the end of the partition). In RANGE mode, CURRENT ROW includes all peers of the current row.

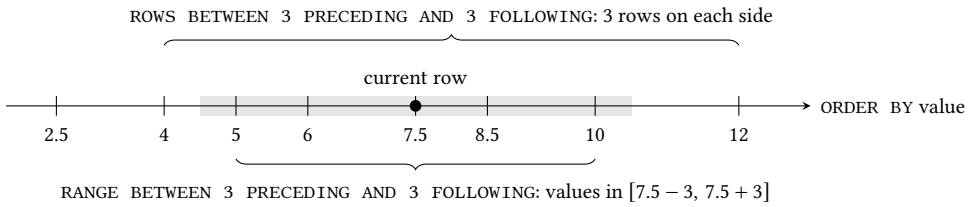


Figure 30: ROWS and RANGE frames around the same row.

When the frame is omitted, a default applies. With ORDER BY it is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW: from the start of the partition to the last peer of the current row. Without ORDER BY it is the whole partition, since all rows are then peers; SUM(amount) OVER () adds 105 to every row of example 7.3.

Aggregates over the frame. Aggregates used as window functions (SUM, AVG, COUNT, MIN, MAX, VARIANCE, STDDEV) are computed over the frame, so the frame clause changes their result: they are **frame-sensitive**. In example 7.3, the running total uses the default frame, which ends at the current row because no customer has two sales on one day, and the 3-row average uses ROWS BETWEEN 2 PRECEDING AND CURRENT ROW: on 4 January customer A gets $(20+15+30)/3 = 21.67$. At the start of a partition the frame is shorter, so A gets 10 on 1 January and $(10+20)/2 = 15$ on 2 January. The frame counts rows, not days: on 4 January customer B's frame still contains the sale of 1 January, because B has no sales in between, and the average is 15. A "3-day moving average" with ROWS is one only if there is exactly one row per day; a RANGE frame with an interval offset counts days, and would give 25.

Ranking and peers. **Ranking functions** number the rows of a partition in their order, ignoring the frame. ROW_NUMBER numbers them $1, 2, 3, \ldots$; peers get different numbers, in an unspecified order. RANK gives peers the same rank and skips the following ranks (gaps), DENSE_RANK gives peers the same rank without gaps. With n rows in the partition, PERCENT_RANK is $(\text{rank}-1)/(n-1)$, the relative rank, and CUME_DIST is the fraction of rows that precede the current row or are its peers. NTILE(k) splits the ordered partition into k buckets of sizes differing by at most one, numbered 1 to k.

Example 7.4 (Ties)**.** Order the four values 30, 20, 20, 10 of a column x by ORDER BY x DESC; the two 20s are peers.

x	ROW_NUMBER	RANK	DENSE_RANK	PERCENT_RANK	CUME_DIST	SUM, default	SUM, ROWS
30	1	1	1	0	0.25	30	30
20	2	2	2	0.33	0.75	70	50
20	3	2	2	0.33	0.75	70	70
10	4	4	3	1	1	80	80

The last two columns compare the default frame with ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. The default frame ends at the last peer, so both 20s see $30 + 20 + 20 = 70$: a running total over a non-unique order jumps over ties. Which of the two 20s gets row number 2, and the sum 50 under ROWS, is unspecified; an additional ordering key makes it deterministic.

Offsets and positions. LAG(*expr*, *offset*, *default*) returns *expr* from the row *offset* positions before the current one in the partition (1 if omitted), and *default* if there is no such row (NULL if omitted); LEAD looks forward in the same way. The day-over-day change of example 7.3 is `amount - LAG(amount)`, which is NULL for the first row of each partition: COALESCE (section 6.3) or the default argument replace it, for instance by a change of 0. Like the frame, LAG counts rows: for customer B it compares 4 January with 1 January.

FIRST_VALUE, LAST_VALUE and NTH_VALUE(*expr*, *n*) return *expr* from the first, last and n-th row of the *frame*, not of the partition. With the default frame, `LAST_VALUE(amount) OVER (PARTITION BY customer ORDER BY day)` returns the current row's own amount (10, 20, 15, 30 for A), not 30 on every row; that requires the frame ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING. So the frame matters for aggregates and for these three functions, while ranking functions and LAG/LEAD use only the partition and its order.

Evaluation order. Window functions are evaluated after FROM, WHERE, GROUP BY and HAVING, and before DISTINCT, ORDER BY and LIMIT. This has three consequences.

- A window function can take an aggregate as argument. Grouping example 7.3 by day gives the daily totals 15, 20, 15, 55, and `SUM(SUM(amount)) OVER (ORDER BY day)` turns them into the running totals 15, 35, 50, 105.
- A window result cannot be used in WHERE or HAVING. To filter on it, compute it in a CTE and filter in the main query. Numbering each customer's sales by decreasing amount with ROW_NUMBER

and keeping the rows numbered at most k gives the top k sales per customer (for $k = 1$: 30 for A, 25 for B), an alternative to the lateral subquery of section 6.2.

- The ORDER BY inside OVER only defines the window; the output order still needs the final ORDER BY.

GROUP BY, CTEs and window functions therefore combine naturally: aggregate the rows, name the result, then annotate each remaining row with running totals, moving averages or ranks.

Named windows. When several functions use the same window, the WINDOW clause defines it once under a name, and each function refers to it with OVER *name*. A moving minimum, average and maximum of a daily series can thus share one centred window, ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING: seven rows per position, fewer near the ends of a partition.

Computing in the database. With these constructs SQL is more than selection, projection and joins: computations such as running totals, rankings or graph traversals run inside the DBMS instead of exporting the data and looping in a general-purpose language such as Python. A graph stored as an edge relation, however, needs recursion to find all people reachable through chains of friendships of any length; whether the relational model is the right fit for such data is a question about data models.

A Command reference

The commands and flags of the command-line tools, grouped by the relational operator they implement (section 1.3).

Streams

`a | b`	stdout of a into stdin of b
`cmd > f`	stdout to file `f`
`cmd < f`	stdin from file `f`
`cmd 2> /dev/null`	discard stderr

Stream ids: 0 stdin, 1 stdout, 2 stderr. `cmd f` = `cat f | cmd`.

Read and inspect

`cat f1 f2`	concatenate to stdout
`cat -v f`	show non-printing bytes
`zcat bzcat xzcat`	decompress to stdout (gzip, bzip2, xz)
`... | less`, `-S`	page through; `-S` chops long lines
`head -n k`	first k lines (also `head -k`)
`tail -n k`	last k lines
`tail -n +k`	from line k on (skip $k-1$)
`head -n -k`	all but the last k (GNU only)
`wc`, `-l`, `-c`	lines, words, bytes; lines only; bytes only

Selection and extraction: grep

`grep 're' f...`	lines matching `re`
`-v`	non-matching lines (negation)
`-i`	ignore case
`-o`	only the matched part, one per line
`-E`	extended regex (`+ ? {}` work)
`-a`	treat binary input as text

Regular expressions

`.`, `\.`	any character; a literal dot
`[a-z0-9_]`, `[^abc]`	one character of / not of a class
`[[:space:]]`, `[[:digit:]]`	whitespace; digit
`r*`	zero or more `r`
`r+`, `r?`	one or more; zero or one (`-E`)
`r{n}`, `r{n,}`, `r{n,m}`	n; $\geq n$; n to m (`-E`)
`^`, `$`	start, end of line
`(r)`, `r|s`, `\1`	group; alternative; back-reference (`-E`)
`\(r\)`, `\{n,\}`	the same in basic syntax

Quote the regex: `'...'`. Greedy: `.*` takes the longest match.

Projection: cut

`cut -f 1,3`	fields 1 and 3 (delimiter: tab)
`cut -f 2-4 -d ','`	fields 2 to 4, comma separated
`cut -c 1-10`	characters 1 to 10

Ignores CSV quoting.

Sorting: sort

`sort`	lexicographic (`10` before `9`)
`-n`	numeric
`-k f`	key from field f (`-k f,f`: only field f)
`-t ','`	field separator
`-u`	drop duplicates

Works beyond main memory.

Grouping: uniq (sorted input!)

`uniq`	one copy per line (distinct)
`uniq -c`	prefix each line with its count
`uniq -d`	lines occurring more than once
`uniq -u`	lines occurring exactly once

Join: join (both inputs sorted on the join field!)

`join f1 f2`	inner join on field 1
`-1 f -2 g`	join field f of `f1`, g of `f2`
`-t ','`	delimiter
`-a 1` / `-a 2`	left / right outer join
`-a 1 -a 2`	full outer join
`-v 1` / `-v 2`	left / right anti-join (unmatched only)

Map: sed

`sed 's/re/new/'`	replace first match per line
`s/.../.../g`	replace all matches
`s/.../.../I`	ignore case (GNU)
`&`, `\1`	in new: whole match, group 1
`-e '...' -e '...'`	several commands
`s:a/b:a_b:`	any delimiter instead of `/`

Aggregation: awk

`awk 'BEGIN { init } { per line } END { final }'`

`$1`, `$2`, ...: fields (whitespace separated; `-F'|'` sets the separator to `|`).

`{ n++ } END { print n }`	count
`{ s+=$2 } END { print s }`	sum of field 2
`{ s+=$2; n++ } END { print s/n }`	average of field 2
`{ if ($1 > 10) s+=$2 }`	conditional sum

Sampling: shuf

`shuf | head -n k`	random sample of k lines

Patterns

count matching lines	`grep 're' f | wc -l`
distinct values	`cut -f 2 f | sort -u`
k most frequent	`sort | uniq -c | sort -n | tail -n k`
lines seen once	`sort | uniq -u`
min / max	`sort -n | head -n 1` / `tail -n 1`
in A but not in B	`sort A > a; sort B > b` `join -v 1 a b`
one word per line	`sed 's/[[:space:]]/\n/g'` (GNU)

Pitfalls

Sort before `uniq` and `join`. Sort counts with `-n`. Without `-E`, `+` is a literal plus. Binary bytes in a text file: `grep -a`. `uniq -u` ≠ distinct.

B Performance tools

Tools for measuring and profiling a program, compiler flags and file-I/O system calls (section 3).

Timing and memory

`time cmd`	*real* (elapsed), *user* (CPU in the program), *sys* (CPU in the kernel)
`free -h`	DRAM and swap: used, free, *buff/cache* (page cache), available; `-h` human units
`sudo sync`	write dirty pages to disk
`sudo sysctl vm.drop_caches=1`	drop the page cache: next run is cold

Run twice: first run cold (from disk), second warm (page cache).

Disk activity and system calls

`vmstat`, `iostat`	watch disk activity (cold run: reads; warm: almost none)
`strace -c cmd`	count and time of each system call (look at `read`)

Counters: perf stat

`perf stat cmd`	task clock, context switches, page faults, cycles, instructions, IPC, branches
`-d`	also L1 and last-level cache loads and misses
`-e ev1,ev2`	only the listed events
`block:block_rq_issue`	event: block I/O request sent to the disk
`block:block_rq_complete`	event: that request completed

Disk reads of a run:

```
perf stat -e block:block_rq_issue,
          block:block_rq_complete cmd
```

Profiles: perf record, scalene

`perf record cmd`	sample where the time goes (writes `perf.data`)
`perf report`	call tree; *Self*: in the function, *Children*: with its callees
`perf record -e ev cmd`	sample on chosen events
`scalene prog.py`	Python: time in Python, in native code, in the system (I/O wait)

Compiling C++

`-O0`	no optimisation
`-O3`	most aggressive optimisation
godbolt.org	Compiler Explorer: source next to generated assembly

File I/O in C/C++

`fd = open(path, O_RDONLY)`	open for reading
`read(fd, buf, n)`	copy *n* bytes into `buf` (from page cache)
`write(fd, buf, n)`	copy into page cache; disk later
`lseek(fd, 0, SEEK_END)`	move to the end; `SEEK_CUR` then returns the size
`mmap(nullptr, size, PROT_READ, MAP_SHARED, fd, 0)`	map the whole file; returns a pointer (or `MAP_PRIVATE`)

First access to each mapped page: page fault. Direct I/O: open with `O_DIRECT`, buffers aligned (e.g. 4 KB).

Pattern: sum of a field, cold and warm

`time awk -F'|' '{ s += $5 } END { print s }' f`
sum of field 5 of a `|`-separated file; run it twice for the cold and the warm time.

C SIMD and multithreading

Compiler flags, vector types, intrinsics, bit tricks and threading constructs (section 4).

Vectorisation reports and assembly

`g++ -O3 -march=native f.cpp`	optimise, use all instructions of this CPU (e.g. AVX2)
`-fopt-info-vec-optimized`	report loops that were vectorised
`-fopt-info-vec-missed`	report loops that were not, and why
`objdump -d prog`	disassemble: check the generated instructions
godbolt.org	source next to assembly, per compiler and flag

Typical *missed* reasons: control flow in loop; iteration count not known; multiple exits.

Reading SIMD assembly (x86)

`xmm`, `ymm`, `zmm`	SIMD registers: 128, 256, 512 bits
`movdqa` / `movdqu`	move packed integers, aligned / unaligned
`movaps`	move aligned packed single-precision (16 bytes)
`paddd`	add packed 32-bit integers (d = doubleword)
`v...` (vpaddd)	AVX encoding of the same instruction

Vector types (GCC)

`typedef int v4si __attribute__((vector_size(16)));`
4 `int`s in 16 bytes; `+ - *` act lane by lane; scalar code where SIMD is missing. Access the lanes via `union { int val[4]; v4si vec; }`.

Intrinsics: names and types

`#include <immintrin.h>`
`_<size>_<op>_<suffix>`: size `mm` (128), `mm256`, `mm512`; op `add`, `sub`, `mul`, `cmpeq`, ...; suffix `ps` float, `pd` double, `epi8/16/32/64` signed, `epu8/16/...` unsigned, `si128/256` whole register.
Types: `__m128i __m256i __m512i` (integer), `__mmask16` (16-lane mask).

Intrinsics used

`_mm_loadu_si128(p)`	load 16 bytes from p (unaligned)
`_mm_storeu_si128(p, x)`	store 16 bytes
`_mm_add_epi32(x, y)`	add 4 32-bit ints
`_mm256_loadu_si256(p)`	load 32 bytes
`_mm256_set1_epi8(c)`	broadcast byte c to 32 lanes
`_mm256_cmpeq_epi8(x, y)`	`0xFF` where bytes equal, else 0
`_mm256_movemask_epi8(x)`	top bit of each byte into an `int`

Find byte c in 32 bytes: `movemask(cmpeq(load(p), set1(c)))`; bit i set $\Leftrightarrow$ byte i = c.

AVX-512 masks (prefix `_mm512_`)

`maskz_add_epi32(k, a, b)`	add selected lanes; others 0
`mask_add_epi32(src, k, a, b)`	add selected; others from `src`
`mask_blend_epi32(k, a, b)`	b where $k = 1$, else a
`maskz_compress_epi32(k, a)`	selected lanes packed to the front
`maskz_expand_epi32(k, a)`	front lanes spread to selected
`compressstoreu`, `expandloadu`	same, to / from memory

Bit i of k controls lane i.

SWAR on 64-bit words

`b * 0x0101010101010101`	byte b in all 8 bytes
`(v - 0x01..01) & ~v & 0x80..80`	`0x80` in zero bytes (lowest one exact)
`~((((v & 0x7F..7F) ^ pat) + 0x7F..7F) & 0x80..80) & (~v & 0x80..80)`	`0x80` in bytes equal to `pat`'s byte (exact)
`countr_zero(m) / 8`	index of the first match (little endian)

OpenMP

`#pragma omp parallel for`	split the loop's iterations among threads
`schedule(static, c)`	fixed chunks of c iterations per thread
`schedule(dynamic)`	threads take the next free chunk
`schedule(guided)`	adaptive chunk size
`reduction(+:sum)`	private partial `sum` per thread, added at the end

Compile with `-fopenmp`.

C++ threads

`thread::hardware_concurrency()`	number of hardware threads
`thread t(f);`	run `f` in a new thread
`t.join();`	wait for it to finish

D SQL reference

Relational operators, query patterns, execution-plan commands, common table expressions and window functions (sections 5 to 7).

Query structure

```
SELECT [DISTINCT] expr AS alias, ...
FROM R r JOIN S s ON r.k = s.k
WHERE row_condition
GROUP BY grouping_columns
HAVING group_condition
ORDER BY expr DESC, other_expr ASC
LIMIT k;
```

The template shows clause order: aliases, joins and the clauses from `WHERE` to `LIMIT` are optional. Square brackets mark optional syntax. `SELECT *` keeps all columns. Table aliases: `R AS r` or `R r`; qualify columns as `r.k`. Default order: `ASC`.

Selection, projection and grouping

`WHERE x < 2 AND y > 0`	row filter, before grouping
`SELECT x, y AS z`	projection, rename
`SELECT DISTINCT x`	remove equal output rows
`GROUP BY x`	one result per value of x
`SUM(x)`, `AVG(x)`	ignore `NULL` inputs
`COUNT(*)` / `COUNT(x)`	rows / non-`NULL` values
`HAVING COUNT(*) > 1`	filter groups after aggregation

Average of qualifying values: `SELECT AVG(x) FROM R WHERE p;`. `AVG` of no non-`NULL` values is `NULL`.

Set operations

`q1 UNION q2`	union, duplicates removed
`q1 UNION ALL q2`	concatenate, duplicates kept
`q1 EXCEPT q2`	rows in `q1` but not `q2`
`q1 INTERSECT q2`	rows common to both

Here `q1` and `q2` are queries with the same number of columns and compatible types. Plain `EXCEPT` and `INTERSECT` also remove duplicates.

Joins

`R JOIN S ON R.k = S.k`	inner, one row per match
`R JOIN S USING (k)`	equality on named common key; one output `k`
`R LEFT [OUTER] JOIN S`	also unmatched left rows
`R RIGHT [OUTER] JOIN S`	also unmatched right rows
`R FULL [OUTER] JOIN S`	unmatched rows on both sides
`R CROSS JOIN S`	all pairs

Outer joins need `ON` or `USING` too; missing-side columns become `NULL`. `FROM R, S WHERE R.k=S.k` is the implicit inner-join form.

Semi-join, anti-join and membership

```
SELECT r.* FROM R r
WHERE EXISTS (SELECT 1 FROM S s
              WHERE s.k = r.k);
```

Use `NOT EXISTS` for the anti-join. The reference to `r.k` correlates the subquery with the current outer row. `SELECT 1` and `SELECT *` test the same existence here.

Membership: `x IN (SELECT k FROM S)`; negation: `NOT IN`. If a nonmatching input meets a `NULL` in the subquery, `NOT IN` is unknown, not true; use `NOT EXISTS` for the no-match pattern.

TPC-H: top ten orders by qualifying revenue

Full query for fig. 24; revenue counts only line items shipped after the cutoff.

```
SELECT l_orderkey,
  SUM(l_extendedprice * (1 - l_discount))
    AS revenue, o_orderdate, o_shippriority
FROM customer
JOIN orders ON c_custkey = o_custkey
JOIN lineitem ON l_orderkey = o_orderkey
WHERE c_mktsegment = 'BUILDING'
  AND o_orderdate < DATE '1995-03-15'
  AND l_shipdate > DATE '1995-03-15'
GROUP BY l_orderkey, o_orderdate,
         o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;
```

`-- comment` ends at the newline. A block comment is `/* comment */`. Strings use straight single quotes; `DATE 'YYYY-MM-DD'` is a date literal.

Pattern: distinct unordered pairs

For `R(id, a)`, pairs of different ids sharing `a`:

```
SELECT DISTINCT r1.id, r2.id
FROM R r1 JOIN R r2 ON r1.a = r2.a
WHERE r1.id < r2.id;
```

The inequality removes self-pairs and reversals; `DISTINCT` removes repetitions due to several shared values.

Inspecting plans

`EXPLAIN SELECT ...;`	estimated execution plan
`EXPLAIN ANALYZE`	execute and report actual statistics, e.g. before `SELECT ...;`

Prepared query with two integer parameters:

```
PREPARE q(int, int) AS
SELECT SUM(bar) FROM test
WHERE id > $1 AND id < $2 GROUP BY foo;
EXPLAIN ANALYZE EXECUTE q(100, 200);
```

`EXECUTE q(100, 200);` runs it without plan output.

Plan fields (PostgreSQL)

`cost=a..b`	estimated startup / total cost, model units
`rows=n`, `width=w`	output rows; bytes per row
`actual time=a..b`	measured startup / completion, ms per loop
`rows=n loops=k`	average rows per loop; executions
`Workers Planned/Launched`	intended / actual parallel workers

Parent costs include children; output row counts are per node. With repeated nodes, multiply average rows or time by `loops` to obtain totals for that node. Read leaves to root; compare estimated and actual row counts. Planning and execution time are reported separately.

Pitfalls

`WHERE` filters rows, `HAVING` filters groups. A join can multiply rows; use `EXISTS` if only presence matters. No guaranteed order without `ORDER BY`. `LIMIT` selects a prefix of that order; ties may need extra ordering keys. `UNION ALL` keeps duplicates. `NULL` is not zero. `EXPLAIN ANALYZE` executes the statement.

SQL reference: advanced constructs

Patterns use `items(id, score)`, `links(id, category, label)` and `categories(category)` (section 6).

Grouping sets, rollup and cube

```
SELECT category, label, COUNT(*) AS n FROM links
GROUP BY GROUPING SETS
  ((category, label), (category), ())
ORDER BY category NULLS LAST, label NULLS LAST;
```

Equivalent bag union:

```
SELECT category, label, COUNT(*) FROM links
GROUP BY category, label UNION ALL
SELECT category, NULL, COUNT(*) FROM links
GROUP BY category
UNION ALL SELECT NULL, NULL, COUNT(*) FROM links;
```

`GROUP BY ROLLUP(a,b)`: `(a,b)`, `(a)`, `()`.
`GROUP BY CUBE(a,b)`: also `(b)`.
`GROUPING(a,b)`: omitted b adds 1, omitted a adds 2. DuckDB also supports `GROUPING_ID(a,b)`.

```
CASE WHEN GROUPING(label)=1 THEN 'subtotal'
     ELSE COALESCE(label,'(none)') END
```

Mask 0 can still contain a data NULL. For a left-joined empty category, `COUNT(l.id)` is 0; `COUNT(*)` counts the padded row.

Statistical and ordered-set aggregates

`STDDEV_POP(x)`, `STDDEV_SAMP(x)`	population / sample
`CORR(x,y)`	Pearson correlation
`COVAR_POP(x,y)`, `COVAR_SAMP(x,y)`	population / sample
`REGR_SLOPE(y,x)`	regression slope
`REGR_INTERCEPT(y,x)`	regression intercept

PostgreSQL: `STDDEV(x)` is `STDDEV_SAMP(x)`.

```
SELECT category,
 PERCENTILE_CONT(0.5) WITHIN GROUP(ORDER BY score),
 PERCENTILE_DISC(0.5) WITHIN GROUP(ORDER BY score),
 MODE() WITHIN GROUP(ORDER BY label)
FROM links l JOIN items i ON i.id=l.id
GROUP BY category;
```

Percentile fraction: $0 \le p \le 1$. `CONT` interpolates; `DISC` returns an input value. Mode ties depend on the engine. DuckDB: `APPROX_QUANTILE(x,p)` gives an approximate quantile.

Subquery shapes and comparisons

Scalar: one column; empty gives NULL, multiple rows give an error.

```
SELECT * FROM items
WHERE id = (SELECT MAX(id) FROM links);
```

Correlated scalar in the `SELECT` list:

```
SELECT l.id, (SELECT i.score FROM items i
              WHERE i.id=l.id) AS score
FROM links l;
```

Maximum as a derived table:

```
SELECT i.* FROM items i
JOIN (SELECT MAX(id) AS m FROM links) t ON i.id=t.m;
```

`x IN (SELECT v FROM R)` is `x = ANY (SELECT v FROM R)`. `x > ALL (SELECT v FROM R)` must hold for every row. Empty: `ANY` false, `ALL` true. `EXISTS` tests row presence. NULL comparisons can yield unknown.

Top value: `ORDER BY id DESC` followed by `FETCH FIRST 1 ROW ONLY` (or `LIMIT 1`).

Count qualifying groups, not input rows:

```
SELECT COUNT(*) FROM (
 SELECT category FROM links GROUP BY category
 HAVING COUNT(*) >= 2) AS t;
```

Collect a sorted array per category:

```
SELECT c.category, ARRAY(SELECT l.label
 FROM links l WHERE l.category=c.category
 ORDER BY l.label) AS labels FROM categories c;
```

No matches: empty array. Element order needs `ORDER BY`.

Decorrelation: anti-join

```
SELECT c.* FROM categories c
WHERE NOT EXISTS (SELECT 1 FROM links l
                  WHERE l.category=c.category);
```

Equivalent using the non-NULL matched identifier:

```
SELECT c.* FROM categories c
LEFT JOIN links l ON l.category=c.category
WHERE l.id IS NULL;
```

Testing a nullable attribute of a matched row is unsafe.

LATERAL: summaries and top-k per group

Lateral subqueries can reference preceding `FROM` items.

```
SELECT c.category, t.n, t.mean FROM categories c
CROSS JOIN LATERAL (
 SELECT COUNT(*) AS n, AVG(i.score) AS mean
 FROM links l JOIN items i ON i.id=l.id
 WHERE l.category=c.category) AS t
ORDER BY t.n, c.category;
```

Ungrouped aggregate: one row even for empty input (count 0, average NULL).

```
SELECT c.category, t.id, t.score
FROM categories c LEFT JOIN LATERAL (
 SELECT i.id, i.score FROM links l
 JOIN items i ON i.id=l.id
 WHERE l.category=c.category
 ORDER BY i.score DESC, i.id LIMIT 1
) AS t ON true;
```

`LIMIT k` gives top-k. `LEFT JOIN` keeps empty categories; `CROSS JOIN LATERAL` drops them. The second ordering key breaks ties.

Strings and regular expressions

Here x, name and `login` are generic text columns.

`x LIKE 'ab%'`	prefix ab, any suffix
`x LIKE '_b_'`	three characters, middle b
`x NOT LIKE pattern`	negate whole-string match
`x ILIKE pattern`	case-insensitive extension
`UPPER(x)`, `LOWER(x)`	case conversion
`SUBSTRING(x FROM 1 FOR 3)`	first three characters

Substring variant: `SUBSTRING(x,1,3)`.
Concatenate: `LOWER(name) || '@cs'`
or `CONCAT(LOWER(name),'@cs')`.
Standard `SIMILAR TO` (support varies):

```
login SIMILAR TO '[a-z]{3}@cs'
```

PostgreSQL-style regex replacement:

```
regexp_replace(login, '@.*$', '')
regexp_replace('ababfooab', '(ab)+', 'xy', 'g')
```

Global flag `'g'` gives xyfooxy. Regex dialects vary.

Conditional expressions and types

```
CASE WHEN score >= 3.9 THEN 'high'
     WHEN score >= 3.0 THEN 'medium'
     ELSE 'low' END
```

First true branch wins; omitted `ELSE` means NULL. Conditional total: `SUM(CASE WHEN p THEN 1 ELSE 0 END)`.
`COALESCE(MIN(label), 'n/a')`: first non-NULL value.
`CAST(AVG(score) AS DECIMAL(3,1))`: explicit conversion, three total digits, one fractional digit; 3.75 ↦ 3.8.

Sampling: specify the dialect

Sampling precedes `WHERE` and aggregation. PostgreSQL (percent):

```
SELECT * FROM R TABLESAMPLE BERNOULLI (20)
REPEATABLE (42);
```

HyPer / DuckDB:

```
SELECT * FROM R
TABLESAMPLE BERNOULLI (20 PERCENT) REPEATABLE (42);
```

`SYSTEM` samples blocks. DuckDB, fixed-size sample:

```
SELECT * FROM R USING SAMPLE reservoir(100 ROWS);
```

DuckDB: `SET threads = 1;` for seeded repeatability. Sampling trades accuracy for work; Bernoulli may still scan the full table.

SQL reference: CTEs, recursion and window functions

Patterns use the relations of section 7:
`students(sid, name)`,
`enrolled(sid, course, grade)`,
`sales(customer, day, amount)`.

Common table expressions

```
WITH t AS (SELECT course, AVG(grade) AS g
           FROM enrolled GROUP BY course)
SELECT course, g FROM t ORDER BY g LIMIT 5;
```

Same result as `FROM (SELECT ...) t`. Header names the columns:

```
WITH c(x, y) AS (SELECT 1, 2)
SELECT x + y FROM c; gives 3.
WITH maxCTE(maxID) AS (
  SELECT MAX(sid) FROM enrolled)
SELECT name FROM students s
JOIN maxCTE ON s.sid = maxCTE.maxID;
```

Without CTE:

```
WHERE s.sid = (SELECT MAX(sid) FROM enrolled).
```

Pipeline, each CTE may read the earlier ones:

```
WITH a AS (...), b AS (SELECT ... FROM a),
     c AS (SELECT ... FROM b)
SELECT ... FROM c;
```

Recursive CTEs

```
WITH RECURSIVE r(cols) AS (
  anchor_query      -- does not read r
  UNION ALL         -- or UNION
  recursive_query   -- reads r = last iteration
) SELECT * FROM r;
WITH RECURSIVE numbers(n) AS (SELECT 1
  UNION ALL SELECT n + 1 FROM numbers WHERE n < 5)
SELECT * FROM numbers;                -- 1 to 5
```

Ancestors in a tree with parent references:

```
WITH RECURSIVE animals(id, name, parent) AS (
  VALUES (1, 'animal', NULL), (2, 'mammal', 1),
  (3, 'giraffe', 2), (4, 'tiger', 2),
  (5, 'reptile', 1), (6, 'snake', 5),
  (7, 'turtle', 5), (8, 'green sea turtle', 7)),
r AS (SELECT * FROM animals WHERE name = 'turtle'
  UNION ALL
  SELECT a.* FROM r JOIN animals a
    ON a.id = r.parent)
SELECT * FROM r;
```

Descendants: anchor on the start row, join `ON a.parent = r.id`.

Reachable nodes in a graph with cycles (edges of fig. 28, each stored once):

```
WITH RECURSIVE friends(a, b) AS (VALUES
  ('Alice', 'Bob'), ('Alice', 'Carol'), ...),
friendship(name, friend) AS (SELECT a, b FROM
  friends UNION ALL SELECT b, a FROM friends),
r(name) AS (SELECT 'Alice'
  UNION
  SELECT f.name FROM r
  JOIN friendship f ON f.friend = r.name)
SELECT * FROM r ORDER BY name;
```

A column growing at each step (a depth d) makes the rows distinct again: bound it in the recursive member, `WHERE r.d < k`.

Window functions

```
f(args) OVER (PARTITION BY p ORDER BY o frame)
```

`ROWS BETWEEN s AND e`	frame by row count
`RANGE BETWEEN s AND e`	by value of the one `ORDER BY` expression
`UNBOUNDED PRECEDING`	partition start
`n PRECEDING`, `n FOLLOWING`	n rows, or values within n
`CURRENT ROW`	this row (`RANGE`: with its peers)
`UNBOUNDED FOLLOWING`	partition end

Default with `ORDER BY`: `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (to the last peer); without `ORDER BY`: the whole partition. `OVER ()`: all rows.

Window function names

Use the frame: `SUM`, `AVG`, `COUNT`, `MIN`, `MAX`, `VARIANCE`, `STDDEV` and

`FIRST_VALUE(x)`	x of the first row of the frame
`LAST_VALUE(x)`	of the last row of the frame
`NTH_VALUE(x, n)`	of the n-th row of the frame

Use only partition and order:

`ROW_NUMBER()`	1, 2, 3, 4; order of peers arbitrary
`RANK()`	peers equal, then gap: 1, 2, 2, 4
`DENSE_RANK()`	no gaps: 1, 2, 2, 3
`PERCENT_RANK()`	$(\text{rank}-1)/(n-1)$, n rows
`CUME_DIST()`	fraction of rows up to the last peer
`NTILE(k)`	bucket $1, \ldots, k$ of near-equal size
`LAG(x, o, d)`	x from o rows before (default 1); d if none (default `NULL`)
`LEAD(x, o, d)`	o rows after

Running total, moving average, change

```
SELECT customer, day, amount,
  SUM(amount) OVER (w ROWS BETWEEN
    UNBOUNDED PRECEDING AND CURRENT ROW) AS run,
  AVG(amount) OVER (w ROWS BETWEEN
    2 PRECEDING AND CURRENT ROW) AS mavg,
  COALESCE(amount - LAG(amount) OVER w, 0)
    AS change
FROM sales
WINDOW w AS (PARTITION BY customer ORDER BY day)
ORDER BY 1, 2;
```

`WINDOW` names a window; `OVER (w ...)` may add a frame to it. `LAG(amount, 1, amount)` also gives change 0 on the first row. `ORDER BY 1, 2` sorts by the first and second output columns.

Aggregate first; filter in a CTE

```
SELECT day, SUM(amount) AS daily,
  SUM(SUM(amount)) OVER (ORDER BY day) AS running
FROM sales GROUP BY day;
```

Top k per group:

```
WITH ranked AS (SELECT s.*, ROW_NUMBER() OVER
  (PARTITION BY customer ORDER BY amount DESC)
  AS rn FROM sales s)
SELECT * FROM ranked WHERE rn <= 3;
```

`RANK` instead of `ROW_NUMBER` keeps all tied rows.

Pitfalls

Window results are not allowed in `WHERE` or `HAVING`: filter in an outer query. `ORDER BY` inside `OVER` does not order the output. The default frame stops at the last peer: running totals jump over ties, and `LAST_VALUE` returns the current row's value unless the frame ends at `UNBOUNDED FOLLOWING`. `ROWS` and `LAG` count rows, not days. `ROW_NUMBER` on ties is arbitrary: add an ordering key. Recursion with `UNION ALL` does not end on a reachable cycle unless a condition bounds it.